

The Pragmatic Programmer

程序员修炼之道

中英对照本

Andrew Hunt

David Thomas

Contents

Preface 前言 (导读)	1
Why “Pragmatic”?	1
Who Should Read This Book?	1
What Makes a Pragmatic Programmer?	2
Individual Pragmatists, Large Teams	2
It’s a Continuous Process	3
How the Book Is Organized	3
What’s in a Name?	3
Source Code, Feedback & Acknowledgments	4
1 A Pragmatic Philosophy	5
1. The Cat Ate My Source Code	5
2. Software Entropy	6
3. Stone Soup and Boiled Frogs	6
4. Good-Enough Software	7
5. Your Knowledge Portfolio	7
6. Communicate!	8
2 A Pragmatic Approach	9
7. The Evils of Duplication	9
8. Orthogonality	10
9. Reversibility	11
10. Tracer Bullets	11
11. Prototypes and Post-it Notes	12
12. Domain Languages	13
13. Estimating	13
3 The Basic Tools	15
14. The Power of Plain Text	15
15. Shell Games	16
16. Power Editing	16

17. Source Code Control	17
18. Debugging	18
19. Text Manipulation	19
20. Code Generators	20
4 Pragmatic Paranoia	21
21. Design by Contract	21
22. Dead Programs Tell No Lies	22
23. Assertive Programming	23
24. When to Use Exceptions	24
25. How to Balance Resources	24
5 Bend, or Break	26
26. Decoupling and the Law of Demeter	26
27. Metaprogramming	27
28. Temporal Coupling	28
29. It's Just a View	29
30. Blackboards	30
6 While You Are Coding	31
31. Programming by Coincidence	31
32. Algorithm Speed	32
33. Refactoring	33
34. Code That's Easy to Test	34
35. Evil Wizards	35
7 Before the Project	36
36. The Requirements Pit	36
37. Solving Impossible Puzzles	38
38. Not Until You're Ready	38
39. The Specification Trap	39
40. Circles and Arrows	40
8 Pragmatic Projects	42
41. Pragmatic Teams	42
42. Ubiquitous Automation	43

43. Ruthless Testing	44
44. It's All Writing	45
45. Great Expectations	47
46. Pride and Prejudice	47
A TIP Cheatsheet TIP 速查卡	49
B Tips Index TIP 总索引	55

Preface 前言导读

前言不长，但把全书的「世界观」交代清楚了：这是一本关于做事（doing）的书，目标是让你成为更好的程序员。下面按原书的小标题逐段导读。

This book will help you become a better programmer.

这本书会帮助你成为一名更好的程序员。

解读 作者一上来就定调：无论你是独立开发者、大团队的一员，还是同时服务多个客户的顾问，这本书都是写给个体的，帮你把工作做得更好。它不空谈理论，而是聚焦实践。作者还做了个词源小考：pragmatic 源自拉丁语 pragmaticus（「精于事务」），再往上溯自希腊语，意为「去做」。所以这是一本关于「去做」的书。

Programming is a craft.

编程是一门手艺。

解读 作者把程序员描述成四种角色的混合体：一部分倾听者、一部分顾问、一部分翻译、一部分「独裁者」。你要在项目时钟的无情滴答声中，捕捉飘忽的需求、写好文档、把工程做扎实——「你每天都在创造小小的奇迹」。紧接着作者泼了盆冷水：工具厂商、方法论大师人人都号称自己的东西是万能解药，但「There are no easy answers.」没有银弹，也不存在普适的「最佳方案」，只有在特定情境下更合适的方案。

Why “Pragmatic”?

为什么叫「务实」

核心论点

务实（pragmatism）意味着：不要死守某一种技术，而要有足够广的背景与经验，好在具体情境下挑出合适的解。理论（计算机科学的基本原理）与实践（各式各样的真实项目）合起来才让你强大。你的做法要随环境和情境不断调整，并在工作推进中持续做这种判断。

解读 这一段是全书价值观的浓缩。作者反对「一招鲜」，也反对纯粹的学院派——强调经验与判断力。一句话概括：「Pragmatic Programmers get the job done, and do it well.」把事情做成，并且做好。

Who Should Read This Book?

谁该读这本书

核心论点

写给想变得更高效、更有产出的程序员。如果你觉得自己没发挥出应有的潜力，或者看着同事用工具比你更高效，或者你当前的工作用的是老技术、想知道新思路怎么用上——这本书就是给你的。作者很坦诚：他们并不掌握全部（甚至不是大部分）答案，想法也不都适用于所有情况；但只要你跟着走，经验会快速增长、效率会提高、对开发全流程的理解会更透彻。

What Makes a Pragmatic Programmer?

什么造就了务实程序员

核心论点

每位开发者都是独一无二的，会形成自己的个人环境。但务实程序员共享以下特质：早期采用者/快速适应者（对新事物有直觉，乐于尝试，上手快）、好奇（爱发问，像囤积癖一样收藏小事实）、批判性思考者（不轻信「一直都是这么做的」，对厂商承诺保持警惕）、务实（努力理解问题的本质，因而对难度和耗时有良好判断）、多面手（尽量熟悉广泛的技术，能随时转向新领域）。最后是两条最基本的、所有务实程序员都具备的特质，作者直接把它们列为 Tip 1 和 Tip 2。

TIP 1

Care About Your Craft

We feel that there is no point in developing software unless you care about doing it well.

TIP 1

关心你的手艺

我们认为，除非你在意把软件做好，否则开发软件毫无意义。

TIP 2

Think! About Your Work

We're challenging you to think about what you're doing while you're doing it. Never run on auto-pilot.

TIP 2

思考！关于你的工作

我们要求你在做事的同时思考你在做的事。永远不要开启「自动驾驶」。

解读 Tip 2 是全书的方法论内核：实时地批判、审视自己的工作，而不是事后做一次性的复盘。作者用 IBM 那句老口号「THINK!」作结，称其为务实程序员的箴言。他们还很诚实地承认这套做法很累——要占用你本就紧张的宝贵时间；回报则是更投入地从事你热爱的工作、对越来越多领域有掌控感，以及持续进步的愉悦。长期看，你和团队会更高效、代码更易维护、开会更少。

Individual Pragmatists, Large Teams

个体的务实主义者，庞大的团队

核心论点

有人主张大团队/复杂项目里没有个人发挥的空间，说「软件构建是一门工程学科，个体自作主张就会崩盘」。作者不同意：软件构建理应是工程学科，但这并不排斥个人的手艺。他们用中世纪大教堂作比——成千上万人年的投入、跨越数十年，木匠、石匠、雕刻匠、玻璃匠都是手艺人，他们在工程要求之上做出了超越纯机械层面的整体。支撑项目走下去的，正是他们对自身贡献的信念。

We who cut mere stones must always be envisioning cathedrals.
—Quarry worker's creed

切割石头的人，心中始终要想着大教堂。——采石工人的信条

解读 作者的判断是：在一百年的尺度上，今天的「工程」也许会被后人看作像中世纪工匠的方法一样古老，而我们的手艺仍将被尊重。换言之，工程规范与个人技艺不是对立的，后者才是长期价值所在。

It's a Continuous Process

这是一个持续的过程

核心论点

访客问伊顿公学的园丁怎么把草坪养得那么完美，园丁答：「每天早上拂去露水，隔天割一次草，每周滚压一次——这样坚持 500 年，你也会有一片好草坪。」优秀的草坪需要少量而持续的照顾，优秀的程序员也一样。作者引入日语词 *kaizen*（改善）：持续做大量小的改进，这是日本制造业质量与生产率大幅提升的主因之一。它同样适用于个人：每天打磨已有技能、给自己的工具箱添点新家伙。

解读 这节和第 1 章的「知识资产」遥相呼应。园丁的段子点出关键：不是靠某次爆发，而是靠微量且不停的投入。作者还给了个乐观的对比——不像伊顿草坪要等 500 年，你的努力几天内就能看到效果，几年后会惊讶于自己的成长。

How the Book Is Organized

本书如何组织

核心论点

全书由一系列短小、自足的小节构成，每节讨论一个主题，彼此有大量交叉引用。可以任意顺序阅读，不必从头读到尾。文中会出现标为 Tip nn 的提示框；作者说这些 tip「有自己的生命」，他们自己每天都在照做。附录 A 汇集资源（参考书目、网址、推荐期刊/书籍/专业组织），附录 B 是练习答案。书中还穿插 exercise（答案较直接）与 challenge（更开放），后者可用于小组讨论或课程作业。

What's in a Name?

名词之辨

核心论点

书里会冒出各种术语，有的是被「技术化」的普通英文词，有的是计算机科学家生造的怪词。作者尽量在首次出现时给出定义或提示，但难免有漏网的。作者给读者的建议很实在：遇到没见过的词别跳过，花点时间查一查（上网或翻教材），查不到就发邮件来吐槽，好让他们在下一版补上定义。

解读 这节还有个有趣的点：作者「反击」了计算机科学家——他们有时会故意不用现成的术语，因为那些术语通常被绑死在某个特定问题域或开发阶段，而本书主张大多数技术在代码、设计、文档、团队组织之间是通用的。现成术语一放到更广的语境就产生歧义，于是作者干脆自造新词。（Humpty Dumpty 那句「一个词的意思就是我选定它是什么意思」是这节的题眼。）

Source Code, Feedback & Acknowledgments

源码、反馈与致谢

核心论点

书中大部分代码取自可编译的源文件，可在 www.pragmaticprogrammer.com 下载，站上还有作者觉得有用的资源链接、勘误与更新。欢迎读者反馈（评论、建议、正文错误、示例问题），邮箱 ppbook@pragmaticprogrammer.com。致谢部分作者感谢了 Addison-Wesley 的编辑与制作团队，以及一长串审阅者——并打趣说，没有这些细致的意见，这本书会更难读、更不准确、还要长一倍。

前言导读完。全书 70 条 TIP 的完整标题对照见附录 A。

A Pragmatic Philosophy

务实的哲学

What distinguishes Pragmatic Programmers? We feel it's an attitude, a style, a philosophy of approaching problems and their solutions.

务实程序员的不同之处在哪？我们认为，是一种态度、一种风格、一种面对问题及其解决方案的哲学。

本章是全书的基石。作者的主张是：务实（pragmatic）不是一套方法论，而是一种思维姿态——永远把眼前的问题放回更大的语境里去看，并对自己的行为 and 结果负责。围绕这一点，本章展开六个话题：责任、软件腐化、推动变革、质量权衡、知识资产，以及沟通。下面逐节导读。

1. The Cat Ate My Source Code

猫吃了我的源码（TIP 3）

核心论点

务实哲学的第一块基石是承担责任：为自己的职业发展、项目和日常工作负责。务实程序员敢于承认自己的无知与错误，而不是把问题推给厂商、语言、管理层或同事。承担责任意味着「主动承诺 + 可被问责」，但你不必接受一个不可能完成、或风险过大的任务——你有权拒绝。

解读 失败是常态：即使测试充分、文档齐全、自动化到位，事情照样会出错——交付延期、冒出未预料的技术难题。关键不在于避免出错，而在于出错时的姿态：诚实、直接，并提供选项而不是借口。作者的判断标准很实用：在你开口解释「为什么做不到」之前，先对着显示器上的橡皮鸭（或猫）把话讲一遍，自己听听这借口是不是很蠢。

The greatest of all weaknesses is the fear of appearing weak.
—J. B. Bossuet, 1709

一切弱点之中，最大的弱点是害怕显得软弱。
—J. B. Bossuet, 1709

实践清单

- 出错时先说「这是我的责任」，再给 2-3 个可行方案，而不是一句「做不完」。
- 接受任务前，先识别自己无法控制的风险；风险过大就明确说明或拒绝。
- 别把问题归给厂商 / 语言 / 管理层——他们的确可能有关，但解决方案得由你给出。

2. Software Entropy

软件的熵 (TIP 4)

核心论点

软件无法免疫于「熵」。当系统里的混乱增加，程序员称之为软件腐化 (software rot)。作者借用犯罪学中的破窗效应：一扇破了很久没人修的窗，会向所有人传递「没人在乎这栋楼」的信号，于是更多窗被打破，涂鸦出现，破坏蔓延，最终积重难返。软件也一样——一个糟糕的设计、一个错误的决定、一段烂代码，如果放着不管，就会拖垮整个项目。

解读 这一节的洞见在于：腐化的加速器是「忽视」。所以务实程序员对破窗的态度是「发现即处理」。哪怕一时没空修好，也要先用木板把洞钉上——把出问题的代码注释掉、显示一个「尚未实现」的提示、或用假数据占位。重点是做出某个动作，表明你掌控着局面，而不是放任它烂下去。作者还讲了个反面故事：一栋豪宅失火，消防员先在地上铺了垫子，因为「不想弄脏地毯」。软件团队有时也犯这种毛病：火在烧，却纠结于无关紧要的体面。

Don't leave "broken windows" unrepaired. Fix each one as soon as it is discovered.

不要任由「破窗」无人修理。发现一扇，就尽快修好一扇。

实践清单

- 巡视自己项目的「街区」，挑出 2-3 个破窗，和同事讨论问题与修法。
- 没时间正式修复时，至少「钉木板」（屏蔽 / 打桩 / 占位），避免继续恶化。
- 警惕「反正这堆代码都是垃圾，我也随便写」的心态——它比破窗本身更致命。

3. Stone Soup and Boiled Frogs

石头汤与温水煮青蛙 (TIP 5、6)

核心论点

石头汤：士兵用一颗「石头」勾起村民的好奇，诱使大家各自拿出胡萝卜、土豆、牛肉……最终煮出一锅人人受益的汤。寓意是：有时你清楚该做什么，但直接申请「做大项目」只会招来委员会的推诿（作者称之为「启动疲劳」）。更好的策略是做催化剂——先做一个小而可用的雏形给人看，再顺势扩展。反面则是温水煮青蛙：对渐进的变化麻木不仁，等察觉时已经晚了。

解读 这两则寓言合起来是一枚硬币的两面。石汤教你如何发起变革：先拿出可运行的东西（而非提案），用既有的成功吸引人们加入。青蛙则提醒你警惕缓慢的侵蚀：大多数软件灾难起初都小到看不见，规格一天天漂移，补丁一层层叠加，士气一点点被磨掉。作者的处方是「Remember the Big Picture」——定期跳出来审视全局，而不只是盯着自己手头那点事。

People find it easier to join an ongoing success.
Show them a glimpse of the future and you'll get them to rally around.

人们更容易加入一场进行中的成功。让他们瞥见未来，他们就会聚拢过来。

实践清单

- 想推动一个改动时，先做出最小可运行的版本，展示它，再提出扩展。
- 定期复盘：项目的规格、代码、进度是不是被「一天一点」地悄悄带偏了？
- 区分两种情形：你是在煮石汤（对所有人都有利），还是在煮青蛙（渐进地伤害对方）？

4. Good-Enough Software

够用就好 (TIP 7)

核心论点

不可能写出完美的软件——时间、技术和人的性情都在与你作对。所以作者主张够好就行 (good-enough)：好到让用户满意、让未来的维护者能接得住、也让你自己心安。但「够好」绝不等于马虎。关键是让用户参与「什么程度算够好」的权衡，并把系统的范围与质量明确写进需求里。

解读 这一节的实践指向是：质量是一个可以被协商和规定的需求项，而不是开发结束时才谈的模糊感受。作者举了「今天的伟大软件常比明天才来的完美软件更可取」这个判断，并提醒：为了加功能或反复打磨而忽视用户诉求是不专业的，但为了赶工期而承诺不可能的时间表、砍掉基本工程底线，同样不专业。分寸就在这两者之间。作者还用绘画作比：不停叠加细节，画会「淹没在颜料里」——要懂得何时停手。

The scope and quality of the system you produce should be specified as part of that system's requirements.

你所产出系统的范围与质量，应当作为该系统需求的一部分被明确规定。

实践清单

- 向用户提问：「你希望这个软件做到多好？」而不是只问「你要什么功能？」
- 把质量目标（容许的缺陷度、打磨程度）写进需求文档，作为可验收项。
- 懂得止损：别让过度的修饰和打磨把好程序毁了。

5. Your Knowledge Portfolio

你的知识资产 (TIP 8、9)

核心论点

你的知识与经验是最重要的职业资产，但它是会贬值的资产。作者把它比作投资组合，并给出五条与理财同构的原则：定期投入、多元化、平衡风险、低买高卖、定期复盘再平衡。其中最重要也最简单的一条是「定期投入」。

解读 第5节大概是本章最著名的一节。作者的具体建议包括：每年至少学一门新语言（不同语言用不同方式解决同一问题，能拓宽思维）、每季度读一本技术书、也要读非技术书（记住计算机是给人用的）、上课、参加本地用户组、尝试不同环境、保持跟进、多看新闻组与网络资源。紧接着的第9条 tip 补上关键一环：批判性地看待你所读所闻——搜索引擎排第一不代表最匹配（可能是付费置顶），书店重点陈列也不代表是好书。

An investment in knowledge always pays the best interest.
—Benjamin Franklin

对知识的投资，永远回报最高。——本杰明·富兰克林

实践清单

- 本周就开始学一门新语言（一直写 C++？试试 Smalltalk 或 Squeak；写 Java？试试 Eiffel）。
- 建立固定的学习节奏：每月/每季度读一本书，跨出当前项目的技术栈。
- 对每一条技术宣传多问一句：谁在从中获利？

6. Communicate!

沟通！(TIP 10)

核心论点

再好的想法、再精妙的代码，如果不能有效地传达给别人，终究是孤儿的想法。开发者一天中的大部分时间都在沟通：开会、访谈用户、写代码、写提案、在团队内游说。作者给出一份实用清单：知道你要说什么（先列提纲）、了解你的听众、选择时机、选择风格、让文档好看、让听众参与、学会倾听、及时回复他人。

解读 这一节的精髓是「沟通是双向的」。作者强调：如果对方没听进去，你就没有在沟通，只是在「说话」而已。了解听众可以用一个助记法 **WISDOM** 来提醒自己（听众要学什么、他们的兴趣点、他们的水平、他们要多细、谁该掌握信息、如何激励他们听）。文档方面作者很务实：内容和呈现都重要，现代工具（ $\text{\LaTeX}$ 、troff）没理由做出难看的文档。最后一条看似琐碎却极重要——及时回复邮件与留言，哪怕只是「稍后答复你」。

Having the best ideas, the finest code, or the most pragmatic thinking is ultimately sterile unless you can communicate with other people.

拥有最好的想法、最精妙的代码或最务实的思维，若不能与他人沟通，终究毫无生机。

实践清单

- 重要会议/重要邮件前，先写提纲，并自问「这样说能传达我真正想说的吗？」
- 说话前先想：听众是谁？他们关心什么？此刻是不是好时机？
- 无论多忙，都回复邮件与语音留言，哪怕只回一句「稍后详复」。

本章导读完。TIP 1-10 的完整标题对照见附录 A。

A Pragmatic Approach

务实的方法

There are certain tips and tricks that apply at all levels of software development, ideas that are almost axiomatic... These approaches are rarely documented as such.

有些技巧适用于软件开发的各个层面，几乎是公理般的道理……但它们很少被当作原则白纸黑字地写下来。

本章把这些散落在设计、项目管理、编码讨论里的通用原则收拢到一起。七个小节两两相关：重复的危害与正交性是一对（前者说别把知识重复，后者说别把同一块知识拆散到多个组件）；曳光弹与原型是一对（都是应对未知，但产物不同）；可逆性讲如何让决策不致命；领域语言讲让代码贴近问题；最后估算讲如何面对有限的时间。

7. The Evils of Duplication

重复的危害（TIP 11、12）

核心论点

知识的不稳定是这一节的出发点：需求会变、法规会变、算法会行不通，所以程序员其实一直处在维护状态，而不是软件发布后才开始维护。而维护最大的敌人是「重复」——同一个知识点在系统里出现多次，改一处就得记得改其余处。作者因此提出全书最著名的原则 **DRY**。

Every piece of knowledge must have a single, unambiguous, authoritative representation within a system.

系统中的每一份知识，都必须有唯一、无歧义、权威的一份表示。

TIP 11

DRY—Don't Repeat Yourself

The alternative is to have the same thing expressed in two or more places. It isn't a question of whether you'll remember: it's a question of when you'll forget.

TIP 11

DRY——不要重复自己

否则同一件事会在两处或多处出现。问题不在于你会不会忘记同步，而在于你什么时候会忘。

解读 作者把重复归成「四个 i」：强加的重复（Imposed，语言/环境似乎逼你重复，比如头文件与实现的重复、文档与代码的重复——对策是写代码生成器、把知识放进元数据，关键是让转换持续自动而非一次性）、无意的重复（Inadvertent，设计失误，比如 Truck 和 DeliveryRoute 都存了 driver，或者在 Line 类里同时存 start/end 和 length——后者应由前两者计算得出）、懒惰的重复（Impatient，赶进度抄一段改改，作者举 Y2K 千年虫为例）、开发者之间的重复（Interdeveloper，最难查——某个州政府审计发现上万份程序各自实现了一遍社保号校验）。对策是加强沟通、设立「项目图书管理员」、把工具脚本集中存放、互相读代码。

TIP 12**Make It Easy to Reuse**

Foster an environment where it's easier to find and reuse existing stuff than to write it yourself. If it isn't easy, people won't do it.

TIP 12**让复用变得容易**

营造一种环境，让找到并复用已有的东西比自己重写更省事。如果复用得费劲，就没人会去复用。

实践清单

- 改任何一处知识时，先问：这份知识在系统里还有别的表示吗？
- 头文件与实现、文档与代码之间不重复同一段说明。
- 把容易重复的规则（校验、格式、常量）集中到一处，或由元数据自动生成。

8. Orthogonality

正交性 (TIP 13)**核心论点**

「正交性」借自几何：两条线成直角即相互独立——沿一条移动，在另一条上的投影不变。在计算里，它表示解耦：两件事正交，就是改动其中一个不影响另一个。良好的系统里，数据库代码与用户界面正交——你换界面不动数据库，换数据库不动界面。

解读 作者用大峡谷直升机讲反面例子：直升机的每个操纵输入都有「次级效应」，压低左手操纵杆会让机头下坠、机身左旋，于是你不得不同时补偿右手操纵杆和脚踏——手脚忙个不停，因为这是个非正交的系统。结论是：「When components of any system are highly interdependent, there is no such thing as a local fix.」高度相互依赖的组件之间，不存在「局部修复」这回事。

TIP 13**Eliminate Effects Between Unrelated Things**

Design components that are self-contained: independent, and with a single, well-defined purpose.

TIP 13**消除不相关事物之间的影响**

把组件设计成自足的：彼此独立，且各自有单一、明确定义的职责。

解读 正交带来两大收益：提高生产力（改动局部化，测试量减少；把 n 件事与 m 件事正交组合能得到 $n \times m$ 的功能，若不正交则相互重叠、结果更少），降低风险（出问题的模块被隔离、便于整块替换；系统更健壮、更易测试、对第三方厂商的依赖被隔离在局部）。作者把正交性应用到团队（衡量标准：一次改动需要多少人参与，人越多越不正交）、设计（分层是强有力的正交手段：问自己「改一个需求会影响几个模块」，理想答案是「一个」）、工具库（警惕引入破坏正交性的库，如 RMI 强制你处理位置相关的异常）、编码（写「害羞的代码」、避免全局数据、避免相似函数）、测试与文档。最后点题：正交与 DRY 相辅相成——DRY 减少内部重复，正交减少组件间相互依赖。

实践清单

- 改一个需求时数一数：牵动了几个模块？目标是「一个」。
- 别用你无法控制的事物的属性做标识（例如拿电话号码当客户 ID）。
- 引入第三方库前问：它是否逼我在代码里做本不该有的改动？

9. Reversibility

可逆性 (TIP 14)

核心论点

工程师和管理层都偏爱「单一、简单的答案」，因为它们方便写进计划表。但现实不配合：今天成立的事实，明天可能就得变。每一次关键决策都会让项目「锁定」到一个更窄的可能性上；当关键决策积累多了，目标就小到一阵风就能吹偏。问题的根源在于：关键决策往往不易逆转。

Nothing is more dangerous than an idea if it's the only one you have. —Emil-Auguste Chartier, 1938

一个想法若成了你唯一的选择，那它最危险。 —Emil-Auguste Chartier, 1938

TIP 14

There Are No Final Decisions

Instead of carving decisions in stone, think of them as written in the sand at the beach.

TIP 14

没有最终决定

不要把决策刻在石头上，把它当作写在沙滩上的字——随时可能有一道浪把它抹掉。

解读 作者的建议是：靠 DRY、解耦、元数据这些手段，减少不可逆决策的数量。比如把「数据库」抽象成「提供持久化服务」，那么从厂商 A 换到厂商 B 就不必大动干戈；把「部署形态」做成配置项，standalone / 客户端-服务器 / n 层就能灵活切换。核心心态是承认「我们第一次未必做出最佳选择」，于是刻意保持灵活——包括在架构、部署、厂商集成这些层面，而不只是代码层面。

实践清单

- 面对关键选择时先问：万一要反悔，代价有多大？能不能把它做成可配置的？
- 用抽象接口把第三方产品「藏」在身后，换厂商时只动一层。
- 架构/部署/厂商集成，也要像代码一样保持灵活。

10. Tracer Bullets

曳光弹 (TIP 15)

核心论点

在黑暗中用机枪射击有两种打法：一种是先精确测定目标位置、环境条件、弹药规格，再用射表算出精确的仰角方位——「一次算准，然后开枪祈祷」；另一种是每隔几发装一颗 曳光弹，它燃烧发光，让你立刻看到弹着点，从而调整瞄准。曳光弹的反馈即时、且与实战环境一致。开发新项目也是如此——你在黑暗中找目标，需求模糊、技术生疏、环境必变。作者推荐用「曳光弹式开发」。

TIP 15

Use Tracer Bullets to Find the Target

Tracer code is not disposable: you write it for keeps. It contains all the error checking, structuring, documentation, and self-checking that any piece of production code has.

TIP 15

用曳光弹寻找目标

曳光代码不是一次性的：你会长期保留它。它包含生产代码该有的一切——错误检查、结构、文档、自检。

解读 关键是端到端打通：把各个组件用最简单但真实的方式连起来，先做出一个「能跑通但功能不全」的骨架

(作者举例：一个能列出整张表的查询，就足以证明 UI ↔ 库 ↔ 服务器这条链路是通的)。之后在每个组件上并行地逐步加实。好处很多：用户能尽早看到东西在动（信心与参与感提升）、开发者有了可依附的结构、随时有一个集成平台（每日多次集成，而非一次性大爆炸集成）、永远有东西可演示、进度更容易衡量。注意曳光弹不保证命中——打偏了就调整瞄准，这正是它的意义。

解读 作者特别澄清「曳光代码」与「原型」的区别：原型是为了探究系统的特定方面，用完后丢弃重写；曳光代码则是精简但完整、构成最终系统骨架的一部分。「Prototyping generates disposable code. Tracer code is lean but complete.」可以理解为：原型是开枪前的侦察与情报收集。

实践清单

- 面对高度不确定的新项目，先做一条端到端打通的「细线」，而不是逐层造好再拼接。
- 用曳光代码当集成平台，每天（多次）集成，而不是攒到最后一刻。
- 分清你是在做「会被丢弃的原型」还是「会保留的曳光骨架」。

11. Prototypes and Post-it Notes

原型与便利贴 (TIP 16)

核心论点

汽车厂商会造很多原型车，各自只测试某个特定方面（空气动力学、造型、结构），因为原型远比量产便宜。软件原型同理：用来分析并暴露风险，以极低成本换取纠错机会。原型不必是代码——便利贴适合给 workflow 和应用逻辑建模，界面可以用白板、画图软件画个假界面。

TIP 16

Prototype to Learn

Prototyping is a learning experience. Its value lies not in the code produced, but in the lessons learned.

TIP 16

用原型来学习

做原型是一次学习经历。它的价值不在产出的代码，而在你学到的东西。

解读 适合拿原型来探究的：架构、既有系统中的新功能、外部数据的结构/内容、第三方工具或组件、性能问题、界面设计。做原型时可以刻意忽略的东西：正确性（可以用假数据）、完整性（只覆盖一条预设输入）、健壮性（可能崩得很难看）、风格（基本没有注释/文档）。作者建议原型用比项目其余部分更高级的语言来写（Perl/Python/Tcl 之类），以便快速拼装。

解读 一个重要的告诫：务必让所有人明白原型是一次性代码。演示得很像模像样的原型很容易误导人，如果预期没设好，管理层可能坚持要把原型（或它的后代）直接上线。作者打了个比方：你可以用轻木和胶带造出很棒的概念车原型，但你不会开它去挤高峰期的车流。若你所在的团队文化很可能误读原型的目的，那么改用「曳光弹」路线更稳妥。

实践清单

- 拿不准的地方，先做原型；明确这次要回答的问题是什么。
- 原型里大胆用假数据、跳过错误处理——但要提前声明这是易耗品。
- 想验证架构时，连代码都可以不写，白板 + 便利贴即可。

12. Domain Languages

领域语言 (TIP 17)

核心论点

语言会影响你如何思考问题，也会扭曲或启发解法——用 Lisp 思维设计和用 C 思维设计，结果不同。更重要的是：问题领域自己的语言往往就暗示了编程解法。作者主张，能贴近领域词汇写代码时就要贴近，甚至进一步用领域的词汇、语法、语义来编程。

The limits of language are the limits of one's world.
—Ludwig Wittgenstein

语言的界限，即世界的界限。——路德维希·维特根斯坦

TIP 17

Program Close to the Problem Domain

By coding at a higher level of abstraction, you are free to concentrate on solving domain problems, and can ignore petty implementation details.

TIP 17

贴近问题领域编程

在更高的抽象层次上编码，你就能专注于解决领域问题，而忽略琐碎的实现细节。

解读 作者展示了用户用自然语言描述的流程，如何直接被整理成一行行「迷你语言」（如 `From X25LINE1 (Format=ABC123) { Put TELSTAR1...}`）。这种语言起初可以只是规约（捕捉需求），也可以进一步真正实现，于是规约就变成了可执行代码。附带好处是领域专属的报错：格式名写错时，不会抛「未声明的标识符」，而是告诉你「“AB123”不是已知格式，已知的有 ABC123、XYZ43B……」——用用户听得懂的话报错。

解读 实现上从简到繁：简单的行式格式用 `switch` 或正则即可；更正式的语法先写 BNF，再交给 yacc/bison 之类的解析器生成器；也可以扩展已有语言（如把应用功能集成进 Python）。作者区分数据语言（产出数据结构，如 `sendmail` 配置、Windows `.rc` 资源文件）与命令式语言（会被真正执行，含语句与控制结构）。最后有个务实的取舍：语法太简单则好解析但难读难扩展（`sendmail` 就是反例）；由于多数应用都会超期服役，作者建议一开始就选更复杂可读的语法，初期投入会在后期维护里连本带利收回。

实践清单

- 留意用户嘴里反复出现的「句式」，它可能就是一门迷你语言的雏形。
- 让报错信息用领域的词汇说话，而不是编译器术语。
- 若应用还要活很久，语法宁愿复杂一点、可读可扩展一点。

13. Estimating

估算 (TIP 18、19)

核心论点

「把《战争与和平》用 56k 调制解调器传完要多久?」「一百万条姓名地址要多少磁盘?」这些问题都缺信息，但如果你会估算，就都能回答——而且在估算的过程中，你会更了解程序所栖居的那个世界。练到对数量级有直觉，你就能一眼判断某个方案是否可行。

TIP 18**Estimate to Avoid Surprises**

By learning to estimate...you will be able to show an apparent magical ability to determine their feasibility.

TIP 18**估算以避免意外**

学会估算之后……你将表现出一种近乎神奇的能力——一眼判断方案是否可行。

解读 多准才算准？先判断答案用于何处：奶奶问「你几点到」是决定做午饭还是晚饭，而水下快没氧气的潜水员要精确到秒。单位会暗示精度——说「大约 130 个工作日」反而显得精确，说「大概半年」则意味着五到七个月都算。作者建议按周期选单位：1–15 天报「天」，3–8 周报「周」，8–30 周报「月」，30 周以上先想清楚再报。

解读 估算从哪来？所有估算都基于对问题的模型。方法链是：先问做过的人 → 弄清被问的到底是什么（包括范围）→ 建一个粗略的心智模型 → 拆成组件、找出各参数如何相互作用 → 给每个参数赋值（重点盯住乘除型的关键参数，加减型的影响小）→ 计算多个版本、给出带条件的答案（如「有 SCSI 总线和 64MB 内存时约 0.75 秒，48MB 时约 1 秒」）。还要记录你的估算，事后对照找差异原因，下一次才会更准。

TIP 19**Iterate the Schedule with the Code**

The only way to determine the timetable for a project is by gaining experience on that same project.

TIP 19**让进度表随代码迭代**

确定一个项目时间表的唯一办法，是在这个项目本身上积累经验。

解读 面对大型项目，常规估算规则会失效。作者主张增量开发，每轮重复：检查需求 → 分析风险 → 设计/实现/集成 → 与用户验证。一开始对「要几轮、每轮多长」只有模糊感觉是正常的——除非你做过几乎一样的项目（同团队、同技术），否则硬要锁定就是瞎猜。做完第一轮后据此修正，每轮都更准，信心随之增长。这招管理层未必喜欢（他们想要开工前就有一个硬数字），但你得帮他们理解：进度由团队、生产率 and 环境共同决定。

What to say when asked for an estimate? You say
“I’ll get back to you.”

被问到估算时该怎么说？你说：「我回头答复你。」

实践清单

- 被要估算时，先问清用途和所需精度，别在茶水间随口报数。
- 报数时用「单位」暗示精度；超过 30 周的项目，先想清楚再开口。
- 记录每次估算与实际的偏差，事后复盘原因。
- 大项目用增量迭代，让进度表跟着代码走。

第 2 章导读完。TIP 11–19 的完整标题对照见附录 A。

The Basic Tools

基础工具

Tools amplify your talent. The better your tools, and the better you know how to use them, the more productive you can be.

工具会放大你的才能。工具越好、你越会用，你的产出就越高。

本章用木匠作比：工匠从一套称手的工具起步，日复一日地打磨、适应，直到工具成为「手的延伸」。程序员也一样——工具是思维通向成品的导管。本章按「原料 → 工作台 → 主力工具 → 保险 → 胶水 → 模具」的顺序，讲了六组基础工具：纯文本（原料）、命令行（工作台）、编辑器、版本控制、调试，以及文本处理与代码生成（胶水与模具）。

14. The Power of Plain Text

纯文本的力量（TIP 20）

核心论点

程序员的「原料」不是木头或铁，而是知识。作者主张：持久保存知识的最佳格式是 纯文本。纯文本不是指没有结构——XML、HTML 都是结构良好的纯文本；它的关键优势是人可读，且不依赖创建它的那个程序。二进制格式的根本问题在于「上下文与数据分离」：数据离开解析它的程序就毫无意义。

TIP 20

Keep Knowledge in Plain Text

With plain text, we give ourselves the ability to manipulate knowledge, both manually and programmatically.

TIP 20

把知识保存在纯文本中

有了纯文本，我们就能同时用手工和 程序两种方式操作知识。

解读 作者点出一个细致的区别：「人可读」不等于「人可理解」。Field19=467abe 每个字符都可打印，但毫无意义；换成 DrawingType=UMLActivityDrawing 才真正自解释。纯文本的三大好处：对抗过时（可读、自描述的数据会比创建它的程序活得久，还只需知道格式的一部分就能解析）、杠杆效应（几乎所有工具——版本控制、diff、grep——都能作用于文本）、便于测试（合成测试数据随手可改，回归输出用 diff 就能比对）。代价是体积更大、解析更慢，但作者认为在异构环境里，纯文本的优势远大于这两点劣势。

实践清单

- 配置、参数、元数据尽量用纯文本存放，别塞进二进制/私有格式。
- 字段名写成自解释的（SSNO 而不是 FIELD10）。
- 担心用户乱改配置时，用校验和/哈希，而不是靠「藏起来」。

15. Shell Games

命令行的游戏 (TIP 21)

核心论点

木匠需要一张结实的工作台；程序员的工作台就是命令行 *shell*。从 *shell* 里你能调用全部工具、用管道把它们组合成原作者都没想到的用法、写宏命令把重复活动自动化。作者反驳「点一点鼠标不也一样吗」：GUI 的优点是所见即所得 (WYSIWYG)，缺点是 所见即全部所得 (WYSIAYG)——它被限制在设计师预想的模型里，而实际工作往往需要越出那个模型。

TIP 21

Use the Power of Command Shells

Gain familiarity with the shell, and you'll find your productivity soaring.

TIP 21

善用命令行的力量

熟悉 *shell* 之后，你会发现自己的效率飞速提升。

解读 作者用几组对比说明命令行的威力，比如「找出比 Makefile 更新的所有 .c 文件」「打包源码」「找出上周没动过的 Java 文件里哪些用了 awt」——*shell* 里一行管道命令的事，GUI 里要点十几下、翻日历、逐个打开文件搜索。命令行「晦涩但强大且简洁」，而且能写成脚本反复使用。当年 Windows 下可用 Cygwin（提供 Unix 兼容层与 120+ 工具）或 UWIN 补齐。

实践清单

- 凡是「点 N 下按钮」的重复操作，想想能否用一行 *shell* 命令或管道搞定。
- 把常用的命令序列固化成脚本，而不是每次重敲。
- 换环境时先摸清有哪些 *shell* 可用，并尽量带上自己惯用的那个。

16. Power Editing

高效编辑 (TIP 22)

核心论点

编辑器尤其体现「工具是手的延伸」——因为文本是编程的基本原料。作者的核心建议是：精通一个编辑器，并用它处理所有编辑任务（写代码、文档、备忘、系统管理、发邮件）。每个环境换一套快捷键和习惯，会让你在任何一处都难以熟练；而统一之后，文本操作会变成肌肉记忆，「按键如歌唱般在文字与思绪间穿行」。

TIP 22

Use a Single Editor Well

Choose an editor, know it thoroughly, and use it for all editing tasks.

TIP 22

熟练使用一个编辑器

选定一个编辑器，彻底掌握它，并用它完成所有编辑任务。

解读 一个好编辑器应有的基本能力：可配置（字体、颜色、快捷键——手不离键盘才高效）、可扩展（新语言、新格式都能「教」给它）、可编程（用宏或内嵌脚本完成复杂多步操作，如 Emacs 的 Lisp）。再加语言相关能力：语法高亮、自动补全、自动缩进、模板、帮助联动、类 IDE 的编译调试。作者很实用地指出：语法高亮不是花

架子——拼错的关键字不按颜色显示，能在你编译之前就跳出来。他还给了张「进阶表」：只用基础功能 → 挑个强大编辑器学好；有偏好但没吃透 → 把功能学全、减少按键；已用得很熟 → 扩展到更多任务。

实践清单

- 选定一个编辑器，逼自己用到熟练；把高频操作都变成不用想的手势。
- 学会你编辑器的「批量」能力（如整段排序、多行编辑），别用记事本硬敲。
- 为反复出现的重复劳动写宏或片段。

17. Source Code Control

版本控制（TIP 23）

核心论点

用户界面里我们最看重「撤销键」——一个原谅错误的按钮。可要是错误发生在上周、你又开关了十次机器呢？这就是版本控制系统的价值：它是一个巨大的撤销键，一台项目级的时间机器，能让你带回「代码还能编译运行」的那个美好时刻。

Those who cannot remember the past are
condemned to repeat it. —George Santayana

不能记住过去的人，注定要重蹈覆辙。——乔治·桑塔亚那

解读 版本控制（SCCS）远不止「撤销」：它能回答「这行是谁改的」「当前版和上周版差在哪」「这个发布改了多少行」「哪些文件最常被改」——这些信息对 bug 追踪、审计、性能与质量分析都极有价值。它还能标记发布版本（随时重建）、管理开发分支（主线继续开发，发布分支修 bug 再合并回主线）。作者的态度非常坚决：「Always.」（永远使用）——哪怕是单人一周项目、哪怕是一次性原型、哪怕不是源码（文档、电话簿、给厂商的备忘、makefile、发布流程，甚至烧 CD 的小脚本）。原因之一是它带来一个隐藏红利：自动且可重复的构建。

TIP 23

Always Use Source Code Control

Even if you are a single-person team on a one-week project. Even if it's a "throw-away" prototype.

TIP 23

始终使用源码版本控制

哪怕你是单人团队、项目只做一周；哪怕这只是一次性的「用完就扔」原型。

实践清单

- 把项目的一切（含文档、脚本、配置）都纳入版本控制。
- 若团队还没用，先给自己建一个私有仓库，别等。
- 借助版本控制做自动、可重复的构建与回归测试。

18. Debugging

调试 (TIP 24–27)

核心论点

没有人能写出完美的软件，所以调试会占据你一天中的很大一部分。作者先谈心态：很多人把调试当成情绪问题（否认、指责、借口、冷漠），但要把它当作一个待解的谜题来攻克。找到了别人的 bug 也别忙着追责——在技术层面，你关心的是修好问题，而不是归咎于人。

TIP 24

Fix the Problem, Not the Blame

It doesn't really matter whether the bug is your fault or someone else's. It is still your problem.

TIP 24

解决问题，而不是归咎于人

bug 是你的还是别人的，其实都不重要——它就是你的问题。

TIP 25

Don't Panic

Resist the urge to fix just the symptoms you see: the actual fault may be several steps removed from what you are observing.

TIP 25

不要惊慌

别急着只修你看到的现象：真正的故障可能离你观察到的位置隔着好几步，还牵涉若干相关之处。

解读 调试的第一条准则是不要惊慌。如果看到 bug 的第一反应是「这不可能」，那你就是错的——别浪费脑细胞在「但这是不可能的」上，因为显然它可能发生，而且已经发生了。

要警惕「短视」：不要只修表象，要设法找到根因。作者还讲了「从一个方向画一笔没事、反方向画就崩」的真实故事，说明两个道理：要采访报 bug 的用户以获取足够细节、人为的测试（只在一个方向上试）覆盖不足。

解读 策略部分：先把 bug 稳定复现（最好能用一条命令复现，而不是 15 步操作）；可视化数据（看程序在操作什么数据，往往比一千句话更有用）；加日志追踪（trace 在并发、实时、事件驱动系统里格外有用，因为那些场景下普通调试器看的是「此刻的状态」，看不到时间维度）；橡皮鸭（对着别人/一只橡皮鸭一步步讲清楚代码该做什么，问题常常自己跳出来——因为讲解逼你把习以为常的假设说出来）；排除法（先怀疑自己的代码，别急着怪 OS 和第三方库）。

TIP 26

“select” Isn't Broken

Remember, if you see hoof prints, think horses—not zebras. The OS is probably not broken.

TIP 26

“select” 没有坏

记住：看到蹄印，先想马、别想斑马。操作系统多半没坏。

解读 TIP 26 来自一个真实教训：一位资深工程师坚信 Solaris 的 select 系统调用坏了，花了几周写绕过方案（而机器上其他网络程序都好好的），最后被迫读文档，几分钟就找到了自己的错误。此后「select is broken」成了团队里的一句自嘲提醒。另外作者指出：如果你「只改了一处」系统就坏了，那多半就是它（哪怕是新版本的 OS、编译器、数据库）。找不到头绪时，可以用经典的二分查找缩小范围。

TIP 27

Don't Assume It—Prove It

The amount of surprise you feel when something goes wrong is directly proportional to the amount of trust and faith you have in the code being run.

TIP 27

别假设——去证明

出事时你有多惊讶，正比于你对那段代码的信任与信仰有多深。

解读 当你被 bug 「吓到」时，说明你的某个假设错了。别因为「我知道这段代码没问题」就跳过它——在本语

境、本数据、本边界条件下证明它。修好之后还要追问：为什么之前没测出来？要不要补测试用例？同类 bug 在别处是否也存在？如果是某个人的错误假设导致的，就在团队里讲开——一个人误解，往往很多人也误解。

实践清单

- 先把 bug 复现成「一条命令」可触发的情形。
- 遇到「不可能」先翻译成「我的某个假设错了」。
- 讲给同事或橡皮鸭听；用二分查找缩小范围。
- 修完追问：测试为何漏掉它？别处是否同病？

19. Text Manipulation

文本处理 (TIP 28)

核心论点

文本处理语言之于编程，好比木工的电木铣——嘈杂、笨重、有点暴力，用错了能毁掉整块料；但在行家手里，它极其强大灵活，只是需要花时间掌握。作者偏爱的有 awk、sed、Perl、Python、Tcl、Ruby 等。用它们可以在半小时里试一个疯狂的想法，而不是花五小时；可以一天就自动化项目的重要环节，而不是花一周。

TIP 28

Learn a Text Manipulation Language

You can quickly hack up utilities and prototype ideas—jobs that might take five or ten times as long using conventional languages.

TIP 28

学一门文本处理语言

你能快速拼出小工具、给想法做原型——同样的活用传统语言可能要花五到十倍的时间。

解读 Kernighan 与 Pike 在《程序设计实践》里用五种语言写同一个程序，Perl 版最短（17 行，对比 C 的 150 行）。作者列举了自己用文本处理语言做过的事：从一份 schema 定义生成 SQL、数据字典、C 库、完整性检查脚本、Web 页面、XML；自动为 Java 属性插入 getter/setter；合并几万条不同格式的测试数据（还顺带查出原数据的不一致）；本书自身的写作（从源文件抽取代码片段、加语法高亮、转成排版语言，从而实践 DRY）；为 C 头文件生成 Object Pascal 接口；生成 Web 文档。

实践清单

- 选一门脚本语言（Perl/Python/Ruby 皆可）练到能随手写小工具。
- 重复的文本转换，写脚本而不是手工改。
- 把「一次性转换」纳入自动构建，避免又变成新的重复源。

20. Code Generators

代码生成器 (TIP 29)

核心论点

木工面对重复劳动时会做夹具 (jig)：夹具做对一次，就能反复复制出同样的活，既省事又减少出错，让工匠专注于质量。程序员也一样——打造一个代码生成器，一旦建成，整个项目周期都能几乎零成本地使用。

TIP 29

Write Code That Writes Code

Once built, a code generator can be used throughout the life of the project at virtually no cost.

TIP 29

写能生成代码的代码

代码生成器一旦建成，就能在整个项目生命周期里几乎零成本地反复使用。

解读 代码生成器分两类。被动生成器只运行一次，产出的代码从此独立、被当作普通源文件编辑和纳入版本控制——它的「出身」会被遗忘；用途包括生成新文件模板、做一次性的语言转换（作者从 troff 转 L^AT_EX 时写的转换器只 90% 准确，其余手工修——被动生成器不必完全准确，你可以自行权衡「完善生成器」与「手修输出」的投入）、生成运行时昂贵的查表。主动生成器在每次需要结果时运行，产出物是可丢弃的（随时能重新生成），这才是真正贯彻 DRY 的关键：让一份知识自动派生出应用需要的所有形态。例如由数据库 schema 自动生成访问代码——schema 一变，代码跟着变，若删掉某列，高层代码会编译失败，你就在编译期而不是用户打电话时才发现问题。作者的经验是：每当你要让两个异构环境协同工作，就该考虑主动代码生成器。

实践清单

- 发现自己在两个地方维护「同一份知识」时，考虑用生成器替代。
- 让派生代码「可丢弃」并纳入构建流程，它就是主动生成器。
- 用被动生成器时接受它「不必 100% 准确」，权衡投入产出。

第 3 章导读完。TIP 20–29 的完整标题对照见附录 A。

Pragmatic Paranoia

务实的偏执

TIP 30

You Can't Write Perfect Software

No one in the brief history of computing has ever written a piece of perfect software. It's unlikely that you'll be the first.

TIP 30

你写不出完美的软件

在计算机短暂的历史上，没有人写出过完美的软件。你也不太可能是第一个。

本章的基调是防御性编程。作者用开车作比：每个人都觉得自己是世上唯一的好司机，所以我们对别人「防御性驾驶」。编码也一样——我们不断与他人的代码打交道、处理可能无效的输入，于是学会了校验、断言、约束。但务实程序员更进一步：连自己也不信任，主动写出防范自身错误的代码。这就是本章五节的主题。

When everybody actually is out to get you, paranoia is just good thinking.
—Woody Allen

当所有人真的都在针对你时，偏执只是一种健全的思考。
—伍迪·艾伦

21. Design by Contract

契约式设计 (TIP 31)

核心论点

人类几千年来解决「不要赖」的办法之一是契约：它界定双方的权利与责任，以及违约后果。既然契约能帮人打交道，能不能用它帮软件模块打交道？能。Bertrand Meyer 为 Eiffel 语言提出了契约式设计 (DBC)：把模块的权利与责任文档化并达成一致，以保证程序正确。何为正确的程序？——不多做也不少于它声称要做的事。

Nothing astonishes men so much as common sense and plain dealing.
—Ralph Waldo Emerson

没有什么比常识和坦荡更让人惊讶的了。 —爱默生

解读 DBC 的三个要素：前置条件 (Preconditions)：调用方必须满足什么，即例程的要求——违反前置条件时例程绝不该被调用，传对数据是调用方的责任；后置条件 (Postconditions)：例程保证做什么，即完成时的状态——有后置条件就意味着例程会终止（不允许死循环）；类不变式 (Class invariants)：从调用方视角看，类始终保证成立的条件——例程内部可以不满足，但返回控制权之前必须重新成立。

If all the routine's preconditions are met by the caller, the routine shall guarantee that all postconditions and invariants will be true when it completes.

只要调用方满足了例程的全部前置条件，例程就保证在完成时让所有后置条件与不变式成立。

TIP 31**Design with Contracts**

Be strict in what you will accept before you begin, and promise as little as possible in return.

TIP 31**用契约来设计**

开始之前，对你所接受的东西要严格；作为回报，承诺得越少越好。

解读 作者把契约和「懒惰的代码」联系起来：如果契约声称「什么输入都接受、回报包治百病」，那你可有的写了。他还讲了契约在继承与多态上的威力——这正是里氏替换原则（Liskov Substitution Principle）的用武之地：「Subclasses must be usable through the base class interface without the need for the user to know the difference.」子类必须能通过基类接口使用，而用户无需知道差别。契约只需在基类写一次，就自动适用于所有未来子类；子类可以接受更宽的输入或做出更强的保证，但至少要接受和保证得跟父类一样多。作者举 `java.awt.Component` 为例：你无法阻止有人写一个「名字对、行为错」的 `setFont`，但有了契约 `@post getFont() == f`，就能保证「你设的字体就是你得到的字体」。

解读 实现层面，作者很务实：**DBC** 最大的价值也许是逼你在设计阶段就正视需求与保证——说清楚输入域、边界条件、例程承诺以及不承诺什么，这本身就是巨大进步；不写清楚，就退回到「靠巧合编程」。没有语言支持时，把契约写成注释也有真实价值。用断言可以部分模拟 DBC，但有几处做不到：无法沿继承层次传播、没有「方法入口时的旧值」概念、运行时库不检查（而问题恰恰常发生在代码与库的边界上）。Eiffel/Sather 等由编译器和运行时自动检查；C/C++/Java 靠预处理器（Nana、iContract）从特殊注释生成校验代码。作者还给了一个漂亮的「语义不变式」例子：借记卡交易系统要求「同一笔交易绝不重复入账」，他们把它写成一句清晰无歧义的规则——*Err in favor of the consumer*（出错了倾向于消费者）。

实践清单

- 设计每个对外方法时，明确写出前置/后置条件与类不变式（哪怕只是注释）。
- 前置条件用来约束「不该发生的事」，不要拿它做用户输入校验。
- 子类只能放宽输入、加强保证，不能反着来。
- 把「绝不可违反」的核心规则提炼成一句无歧义的语义不变式，写进文档。

22. Dead Programs Tell No Lies

死程序不说谎（TIP 32）**核心论点**

别人有时比你更早发现你状态不对；代码也一样。一个跑偏的指针可能覆盖了文件句柄，下一次 `read` 才暴露；缓冲区溢出可能毁掉了计数器，直到 `malloc` 失败。我们很容易陷入「这不可能发生」的心态，从而漏掉本该检查的错误。作者的主张是：只要出现错误，就说明一定发生了非常糟糕的事。

TIP 32**Crash Early**

A dead program normally does a lot less damage than a crippled one.

TIP 32**尽早崩溃**

一个死掉的程序，通常比一个瘸着的程序造成的损害小得多。

解读 「崩溃，而不是带病运行」（Crash, Don't Trash）是这一节的核心。继续运行的代价可能是把损坏的数据写进关键数据库、让洗衣机连续甩干二十次。Java 的运行时在这点上做得很好：意外发生时抛 `RuntimeException`，

未被捕获就一路冒到顶层、打印堆栈并终止。没有异常机制的语言（如 C）可以用宏包装「绝不该失败」的调用，失败就打印文件/行号并退出。作者承认有时不能直接退出（要释放资源、写日志、回滚事务），但基本原则不变：当代码发现「本该不可能」的事发生了，程序已不再可靠，此后它做的一切都可疑，所以尽快终止它。

实践清单

- 每个 case/switch 都写 default，用于捕捉「不可能」。
- 发现「本该不可能」的状态，宁可立即崩溃，也别带着脏数据继续跑。
- 崩溃前若需善后（释放/回滚），把善后逻辑集中写好，别变成常规流程。

23. Assertive Programming

断言式编程（TIP 33）

核心论点

每个程序员早年都被灌过一句咒语：「这（里）绝不可能发生」——「这段代码 30 年后早不用了，两位年份够了」「这程序不会用到国外，国际化干啥」「count 不可能是负的」。作者说：别自欺，尤其在写代码的时候。

TIP 33

If It Can't Happen, Use Assertions to Ensure That It Won't

Whenever you find yourself thinking "but of course that could never happen," add code to check it.

TIP 33

若不可能发生，就用断言确保它不会

每当你冒出「这当然不可能发生」的念头时，就加一段代码去检查它。

解读 断言（assert）检查一个布尔条件。两条铁律：断言里的条件不能有副作用（否则你会造出改了行为才复现的 Heisenbug——作者举了在 ASSERT 里调用 nextElement() 导致循环只处理一半元素的例子）；断言不代替真正的错误处理——不要用它校验用户输入（assert(ch=='Y' || ch=='N') 是坏主意）。另外，断言可能被编译关闭，所以绝不要把必须执行的代码放进 assert。

解读 作者特别反对「上线就关断言」的常见做法。这种说法有两个错误假设：一是以为测试能覆盖所有 bug（复杂程序连极小比例的排列都测不完），二是忘了程序跑在一个危险的真实世界里——测试时不会有老鼠咬断电缆，但生产环境里日志可能写满硬盘。他打了个绝妙的比方：「Turning off assertions...is like crossing a high wire without a net because you once made it across in practice.」——因为练习时走过去过，就走钢丝不挂网，戏剧性十足，但很难买到人寿保险。真有性能问题，也只关掉确实拖慢你的那几条断言。

实践清单

- 把「这不可能」逐条改写成一句断言。
- 确保断言条件无副作用，且不承担真实错误处理。
- 交付时保留断言；若必须关，只关真正影响性能的那几条。

24. When to Use Exceptions

何时使用异常 (TIP 34)

核心论点

「检查每个可能的错误」在实践中会让正常逻辑被错误处理淹没——作者贴了一段层层嵌套 `if/else` 的代码，正常流程几乎看不见。用异常重写后，「正常控制流」变得清晰，所有错误处理被收拢到一处。问题在于：什么才算「异常」？

TIP 34

Use Exceptions for Exceptional Problems

An exception represents an immediate, nonlocal transfer of control—it's a kind of cascading goto.

TIP 34

用异常处理异常问题

异常是一种立即的、非局部的控制转移——它是一种级联的 `goto`。

解读 作者的判据很实用：异常应当很少用于程序的正常流程，应留给意外事件。检验方法是假设「未捕获的异常会终止程序」，然后问自己：若把所有异常处理器都去掉，这段代码还能正常跑吗？若答案是否定的，说明异常被用在了非异常的场景。以「打开文件失败」为例，答案取决于语境：`/etc/passwd` 本应存在，打不开就是异常（意外）；而用户从命令行指定的文件，本就不知道是否存在，找不到不算异常，应返回错误码。正因为异常是「级联 `goto`」，把它当常规流程用会带来意大利面条式的可读性/可维护性问题，还会破坏封装——例程与调用方因异常处理而耦合更紧。

实践清单

- 异常只留给「意外」；可预期的失败用返回码。
- 用「去掉所有 handler 还能正常跑吗」来检验是否滥用了异常。
- 别让异常成为跨越正常控制流的手段。

25. How to Balance Resources

如何平衡资源 (TIP 35)

核心论点

我们写代码时都在管理资源：内存、事务、线程、文件、定时器——都是有限的东西。多数时候用法有固定模式：分配 → 使用 → 释放。但很多开发者没有一致的计划。作者给出一个简单原则：谁分配，谁释放。

TIP 35

Finish What You Start

The routine or object that allocates a resource should be responsible for deallocating it.

TIP 35

有始有终

分配资源的例程或对象，应当负责释放它。

解读 作者用反面例子说明：`readCustomer` 打开文件并把句柄存进全局变量，`writeCustomer` 用它并在末尾关闭——两个例程通过全局变量紧耦合。当需求变成「只在余额非负时更新」后，某条路径不再调用 `writeCustomer`，文件就永远不关，生产环境跑几小时就「打开文件过多」而崩。把关闭动作挪到调用方是治标不治本（变成三个例程耦合）。正确的重构是把文件句柄作为参数传递，由最外层的 `updateCustomer` 负责 `open` 与 `close`——`open` 和 `close` 出现在同一处，「每次 `open` 必有对应 `close`」一目了然，全局变量也消失了。

解读 嵌套分配两条补充规则：按相反顺序释放（避免一个资源还引用着另一个时被孤立）；在代码中多处分配同一组资源时，始终按相同顺序分配（降低死锁风险——A 占住 r1 等 r2，同时 B 占住 r2 等 r1，两者就会永远互等）。这个模式适用于任何资源。面向对象语言里，可以把资源封装进类：构造获得、析构释放。C++ 里异常会干扰资源释放（正常路径和 catch 路径各删一次 n，违反 DRY），此时利用「局部对象出作用域自动析构」的特性——把指针换成栈上的对象即可。

实践清单

- 谁 open 谁 close：分配与释放写在同一个作用域里。
- 多资源时：逆序释放；多处分配则统一顺序，防死锁。
- 面向对象语言：用构造/析构（或 RAII）自动平衡资源。
- 别用全局变量在例程之间传递资源句柄。

第 4 章导读完。TIP 30–35 的完整标题对照见附录 A。

Bend, or Break

弯曲，或折断

Life doesn't stand still. Neither can the code that we write...we need to make every effort to write code that's as loose—as flexible—as possible.

生活不会静止不动，我们写的代码也不能。……我们要尽力让代码尽可能松、尽可能灵活。

第2章讲过「不可逆决策」的危险；本章讲如何做出可逆的决策，让代码在不确定的世界里保持柔韧。路线是：先看耦合（迪米特法则），再学少写代码（元编程）、时间维度上的解耦（时间耦合）、模型与视图分离，最后更是更彻底的解耦（黑板系统）。目标：写出能「随机应变」的代码。

26. Decoupling and the Law of Demeter

解耦与迪米特法则（TIP 36）

核心论点

「害羞的代码」是双向的：不向别人暴露自己，也不跟太多人打交道。作者用间谍组织的「细胞」作比：细胞之间互不知晓，一个细胞暴露也不会牵连其他人。代码也一样——按模块分「细胞」，限制彼此往来，某个模块失效被替换，其余模块照常运转。

Good fences make good neighbors. —Robert Frost, “Mending Wall”

好篱笆造就好邻居。

—罗伯特·弗罗斯特

解读 作者用「总承包商」类比：你雇了总承包商，就不必自己去跟各个分包商打交道。软件里也一样——你请求一个对象提供服务，就别让它再甩给你一个第三方对象让你自己去处理。反面例子很典型：为了画图，一路 `aSelection.getRecorder().getLocation().getTimeZone()` 挖下去，就把绘图例程无谓地耦合到了 `Selection`、`Recorder`、`Location` 三个类上；一旦 Fred 改了 `Location`，你的代码也得跟着改。正确做法是直接要你需要东西：让 `Selection` 提供一个 `getTimeZone()`，绘图例程不关心时区从哪里来。作者列出依赖爆炸的三个症状：链接一个单元测试的命令比测试程序本身还长；对某模块的「简单」改动在无关模块间传播；开发者不敢改代码，因为不确定会牵连什么。

TIP 36

Minimize Coupling Between Modules

The Law of Demeter for functions...tries to prevent you from reaching into an object to gain access to a third object's methods.

TIP 36

最小化模块间的耦合

函数版迪米特法则……力图阻止你「伸手」进入一个对象，去访问第三个对象的方法。

解读 迪米特法则的核心规则：一个对象的方法，只应调用——它自身、传进来的参数、它创建的对象、它直接持有的组件对象——的方法。研究 ([BBM96]) 表明：C++ 中「响应集」更大的类更容易出错（响应集 = 该类

方法直接调用的函数数量)，遵循迪米特法则能缩小响应集，从而减少错误。代价是：你得像总承包商一样，写大量只是转发请求的包装方法，带来运行时和空间开销；在性能敏感处（如同数据库的「反范式化」）可以权衡——只要耦合被明确知晓且可接受，设计就没问题。作者还提了「物理耦合」：文件、目录、库之间的依赖（John Lakos 《大规模 C++ 软件设计》），逻辑设计与物理设计必须同步进行。

实践清单

- 别链式调用 `a.getB().getC().doSomething()`；让 `a` 替你代办。
- 给被频繁穿透的对象补上「转发方法」，把依赖收口到一处。
- 物理层面（文件/库依赖）也要控制，避免构建周期以天计。

27. Metaprogramming

元编程（TIP 37、38）

核心论点

细节会毁掉我们干净的代码——尤其当细节频繁变化时。每次为了顺应业务、法规或管理层一时口味去改代码，都在冒引入新 bug 的风险。所以作者说：「把细节请出去！」把它们挪出代码，让程序变得高度可配置、柔软。

No amount of genius can overcome a preoccupation with detail.
—Levy's Eighth Law

再高的天才，也敌不过对细节的执迷。——莱维第八定律

TIP 37

Configure, Don't Integrate

Tuning parameters, user preferences, the installation directory...should be implemented as configuration options, not through integration or engineering.

TIP 37

配置，而非集成

调优参数、用户偏好、安装目录……都应作为配置项实现，而不是靠集成或工程手段。

解读 关键词是元数据——「关于数据的数据」。作者取最宽泛的定义：任何描述应用的数据（该如何运行、用哪些资源）。它通常在运行时读取，而非编译期。浏览器隐藏工具栏的偏好、Windows 的 .ini/注册表、Unix 的 X 资源文件、Java 的 Property 文件都是元数据；更强大的做法是内嵌一门脚本语言。

TIP 38

Put Abstractions in Code, Details in Metadata

Program for the general case, and put the specifics somewhere else—outside the compiled code base.

TIP 38

抽象进代码，细节进元数据

为一般情形编程，把具体细节放到别处——编译产物之外。

解读 这样做的好处：迫使你解耦、推迟细节从而得到更抽象健壮的设计、不必重新编译就能定制（甚至能给线上系统的严重 bug 提供应急变通）、元数据可以表达得比通用语言更贴近问题领域、同一个「引擎」配不同元数据能支撑多个项目。作者特别强调业务逻辑/规则最适合放进灵活的格式（规则系统、迷你语言），因为它们变化最频繁。还有个实用问题：程序何时读配置？很多程序只在启动时读一次，改配置就得重启——长驻服务应考虑运行中重载。最后用渡渡鸟（dodo）作结：不能适应的物种会灭绝，别让你的项目（或职业）也走上这条路。

实践清单

- 把「会变的东西」（算法、数据库、界面风格、业务规则）做成配置项。

- 为一般情形写引擎，把具体情形放进元数据/迷你语言。
- 长驻服务支持运行中重载配置，别动不动就重启。

28. Temporal Coupling

时间耦合 (TIP 39–41)

核心论点

时间是被软件架构忽视的维度——这里说的不是日程，而是时间作为设计元素。它有两面：并发（同时发生）与顺序（时间上的相对位置）。人们设计和编程时习惯线性思维：「先做这个，再总做那个」。这会带来时间耦合：A 必须在 B 之前被调用、一次只能跑一个报表、得等屏幕重绘完才能收到点击——「滴答必须在嗒之前」。这不灵活，也不现实。

TIP 39

Analyze Workflow to Improve Concurrency

You can use activity diagrams to maximize parallelism by identifying activities that could be performed in parallel, but aren't.

TIP 39

分析工作流以改善并发

用活动图找出那些本可并行、实际却串行的活动，从而最大化并行度。

解读 工作流：用 UML 活动图把用户描述的工作流画出来会很开眼——用户往往按线性顺序描述（比如调一杯 piña colada 的 12 步），但活动图会暴露真正的依赖：「开搅拌机」「开椰浆」「量朗姆酒」「拿杯子」「拿小粉伞」其实可以同时做。架构：作者讲了一个 OLTP 系统，做成三层多进程分布式，靠工作队列通信、请求全异步，从而简洁又高性能。由此引出饥饿消费者模型（hungry consumer）：用「一组独立消费者 + 一个集中工作队列」代替中心调度器，谁饿了自己去队列抓活干；某个任务卡住时别人能顶上，各自按自己节奏推进——这就是时间解耦。

TIP 40

Design Using Services

Instead of components, we have really created services—independent, concurrent objects behind well-defined, consistent interfaces.

TIP 40

用服务来设计

我们真正创建的不是组件，而是服务——独立、并发的对象，藏在定义良好且一致的接口之后。

TIP 41

Always Design for Concurrency

Concurrency forces you to think through things a bit more carefully—you're not alone at the party anymore.

TIP 41

始终为并发而设计

并发逼你把事情想得更周全——派对上不再只有你一个人。

解读 即使不做多线程，也应按并发的约束来设计，因为那些约束很有益：全局/静态变量必须防止并发访问（于是你会反省：我为什么需要全局变量）；对象必须在被调用的任何时刻都处于合法状态（别再依赖「创建后到显示前没人会碰它」这种巧合）。作者用 strtok 与 Java StringTokenizer 对比：strtok 在线程不安全之外，还有讨厌的时间依赖（第一次调用必须传待解析串，之后必须传 NULL，且内部保留隐式状态，无法同时解析两个串）；而 StringTokenizer 的接口干净、无隐式状态、可同时解析多个串。思考并发与时间依赖，能反过来帮你设计出更干净的接口。好处不止于此：一旦架构里有了并发的元素，「部署形态」也变得灵活（standalone

/ 客户端-服务器 / n 层随时可配)。反过来（给非并发应用事后加并发）要难得多。

实践清单

- 用活动图找出可并行却被串行化的步骤。
- 用「工作队列 + 独立消费者」解耦处理节奏（饥饿消费者模型）。
- 保证对象在任何被调用时刻都处于合法状态，别依赖调用时序。
- 警惕带隐式状态、要求特定调用顺序的接口（反面教材 strtok）。

29. It's Just a View

这只是一个视图（TIP 42）

核心论点

把程序拆成模块后，新问题来了：运行时对象怎么互相通信？如何管理它们之间的逻辑依赖、同步各自的状态变化？要做得干净灵活——不希望它们彼此知道太多，让每个模块「只听到它想听的东西」。作者引入事件：一条「刚发生了件有意思的事」的特殊消息，用来通知别的对象。这样能把对象间的耦合降到最低——发事件的人无需知道接收者是谁。

Still, a man hears what he wants to hear and disregards the rest. —Simon and Garfunkel, “The Boxer”

人总是只听自己想听的，其余的充耳不闻。——西蒙与加芬克尔《拳击手》

解读 作者先批评一种坏做法：让一个例程接收所有事件（早期 Java 如此）——它违反了封装（那个例程得了解众多对象间的交互）、增加了耦合，还常导致巨大的 case 或多路 if。正确做法是发布/订阅（publish/subscribe）：对象只注册自己关心的事件，事件也只送给需要它的对象——「别给对象发垃圾邮件」。Publisher 维护订阅者名单，事件发生时依次通知。

TIP 42

Separate Views from Models

By loosening the coupling between the model and the view/controller, you buy yourself a lot of flexibility at low cost.

TIP 42

让视图与模型分离

放松模型与视图/控制器之间的耦合，你能以很低的代价换来极大的灵活性。

解读 由此引出 MVC（模型-视图-控制器）。以电子表格为例：数据本身是模型，表格、柱状图、合计框是不同视图，缩放/平移是控制器——三份数据不能各存一份。分离之后你能：同一模型有多个视图、通用视图服务多个模型、多个控制器提供非常规输入。作者以 Java 树控件（JTree/TreeModel/TreeCellRenderer）为例：你只需提供一个符合 TreeModel 的数据源，就得到一个完整可导航的树；老板要求给某些员工换个「骷髅头」图标时，只改 TreeCellRenderer 即可。他还强调 MVC 不止用于 GUI：视图只是对模型的一种解释（未必图形化），控制器是一种协调机制（未必对应输入设备）。棒球转播的例子展示了「模型-视图网络」：比分、球员、记录、天气各是一个模型，解说词、电视字幕、网页各是订阅它们的视图；视图又能再成为更高层对象的模型——因为是网络而非线性链条，灵活性极大。

实践清单

- 用发布/订阅而非「一个例程收所有事件」。
- 数据（模型）与展示（视图）分开，别各存一份数据。
- 让视图/控制器可替换、可叠加（同模型多视图，同视图多模型）。

- MVC 不限于 GUI——任何「解释」都是视图，任何「协调」都是控制器。

30. Blackboards

黑板 (TIP 43)

核心论点

侦探如何协作破案？首席探员在会议室立一块大黑板，写下一个问题——「H. Dumpty（男性，蛋）：意外还是谋杀？」——各位侦探各自往板上贴事实、证词、物证、推测。没有哪个侦探需要知道其他侦探的存在，他们只是盯着板子找新信息、再添上自己的发现。这些人可能来自不同专业、不同警局，唯一的共同点是想破案。

TIP 43

Use Blackboards to Coordinate Workflow

A blackboard system lets us decouple our objects from each other completely, providing a forum where knowledge consumers and producers can exchange data anonymously and asynchronously.

TIP 43

用黑板协调 workflow

黑板系统让我们把对象彻底解耦，提供一个论坛，让知识的消费者与生产者匿名、异步地交换数据。

解读 黑板系统的关键特性：参与者互不知晓、可来自不同「专业」、可随时进出、板上放什么没有限制（图片、句子、物证都行）。技术渊源是 AI 领域（语音识别、知识推理），现代实现如 JavaSpaces、T Spaces 基于 Linda 的元组空间（tuple space）思想：以键/值对存活对象（不只是数据），通过模板与通配符做部分匹配来检索——例如用 `Author` 模板 + `lastName="Shakespeare"`，能捞出作者莎士比亚，而不是园丁莎士比亚。主要操作有 `read` / `write` / `take`（读并移除）/ `notify`（匹配写入时自动通知）。因为能存对象，黑板不仅能做「数据流」，还能做「对象流」的算法设计。最大好处是单一、一致的接口——而传统分布式应用常要为每个交互手工设计 API，界面组合爆炸会很快变成噩梦。作者最后用抵押贷款申请为例：法律规则复杂，用「黑板 + 规则引擎」很优雅——数据到达顺序无关紧要，一条事实贴上去就能触发相应规则，规则输出也可再贴回黑板触发更多规则。

实践清单

- 需要多方匿名、异步地交换信息时，考虑黑板模型。
- 用统一的「黑板接口」替代为每个交互定制的 API。
- 把黑板与规则引擎结合，让数据到达顺序变得无关紧要。
- 大型黑板要分区（扁平兴趣组或树状结构），避免杂乱难找。

第 5 章导读完。TIP 36–43 的完整标题对照见附录 A。

While You Are Coding

编码之时

Coding is not mechanical. If it were, all the CASE tools that people pinned their hopes on in the early 1980s would have replaced programmers long ago.

编码不是机械劳动。若真是，那 1980 年代初人们寄予厚望的那些 CASE 工具，早就该取代程序员了。

流行的看法是：项目一旦进入编码阶段就基本是机械活——把设计「誊写」成可执行语句。作者认为这种态度正是许多程序丑陋、低效、结构差、难维护甚至根本不对的最大原因。编码时每分钟都有需要仔细判断的决策。本章讲五件事：不要靠巧合编程、估算算法速度、重构、让代码易于测试、提防你不理解的「邪恶向导」。

31. Programming by Coincidence

靠巧合编程 (TIP 44)

核心论点

老战争片里，疲惫的士兵用刺刀探前方有没有地雷——探了几下没炸，就以为安全了，挺直腰板大步往前走，结果被炸得粉碎。他的探雷什么也没证明，只是运气好而已，却让他得出了错误结论。我们开发者也在雷区里工作：不要靠运气和偶然的成功，要刻意地编程。

Fred doesn't know why the code is failing because he didn't know why it worked in the first place.

Fred 不知道为什么代码坏了，因为他从一开始就不知道它为什么能工作。

TIP 44

Don't Program by Coincidence

We should avoid programming by coincidence—relying on luck and accidental successes—in favor of programming deliberately.

TIP 44

不要靠巧合编程

要避免靠巧合编程——依赖运气和偶然的成功——转而刻意地编程。

解读 巧合的几种来源：实现上的偶然——依赖未文档化的错误或边界行为；你按某例程对坏数据的反应写代码，作者却从没打算那样工作，等他「修好」例程，你的代码就崩了。作者举了连串重绘调用（`paint`、`invalidate`、`revalidate`、`repaint` 之类）的例子——那些例程从没被设计成那样调用，能用纯属巧合；而且事后 Fred 还舍不得把这些多余调用删掉（「能用就行」，这句心态很危险：它可能只是看起来能用；边界是偶然的；未文档化行为可能随库的下个版本改变；多余调用让代码更慢、还引入新 bug）。语境上的偶然——你是给 GUI 写的工具模块，就非要依赖 GUI 存在吗？依赖用户说英语吗？隐式假设——需求、测试各层面都充满未记录的假设，且开发者之间常相互冲突。

解读 怎么「刻意」编程？时刻清楚自己在做什么（别像温水里的青蛙）；别蒙着眼睛写代码（不熟悉的技术/领域是巧合的温床）；从计划出发；只依赖可靠的东西；把假设写下来（契约式设计）；连假设也一起测试——不

确定就真去试，用断言验证，假设对了就是给代码加了文档，假设错了就庆幸自己发现了；分清主次（先把基础设施做对，花哨功能才不至于白搭）；不要被历史绑架（现有代码不合适就该替换）。

实践清单

- 遇到「能跑但不知道为什么」的代码，先搞清楚原因再前进。
- 只依赖有文档的行为，别依赖偶然的边界表现。
- 把假设写成断言并测试它；不确定就实际跑一遍。
- 清掉那些「反正能用」的多余调用。

32. Algorithm Speed

算法速度 (TIP 45、46)

核心论点

除了估算走路/项目要多久，务实程序员还几乎每天估算另一件事：算法消耗的资源（时间、处理器、内存）。在两种做法之间怎么选？程序处理 1000 条记录要多久你知道，但扩展到 100 万条呢？哪些代码需要优化？这些问题往往能用常识 + 一点分析 + 一种近似写法（「大 O」记号）来回答。

Think of the O as meaning “on the order of.” The O notation puts an upper bound on the value of the thing we’re measuring.

把 O 理解为「在……量级上」。O 记号为我们所度量的东西（时间、内存等）给出一个上界。

解读 大 O 只保留最高阶项、忽略常数因子——这是它的缺点：两个都是 $O(n)$ 的算法可能相差一千倍，但记号上看不出来。常见量级： $O(1)$ 常数、 $O(\lg n)$ 对数（二分查找）、 $O(n)$ 线性（顺序查找）、 $O(n \lg n)$ （快排/堆排平均）、 $O(n^2)$ 平方（选择/插入排序）、 $O(n^3)$ 立方（矩阵乘法）、 $O(2^n)$ 指数（旅行商、集合划分）。作者给了个直观的换算：若 $O(n)$ 处理 100 条要 1 秒，处理 1000 条就是 10 秒；而 $O(n^2)$ 要 100 秒；指数级的恐怕要等到「咖啡煮好、宇宙终结」。

TIP 45

Estimate the Order of Your Algorithms

Whenever you find yourself writing a simple loop, you know that you have an $O(n)$ algorithm. If that loop contains an inner loop, then you’re looking at $O(n^2)$.

TIP 45

估算算法的量级

每当你写一个简单循环，你就有了一个 $O(n)$ 算法；若循环里还套着循环，那就是 $O(n^2)$ 。

解读 用常识就能估出很多算法的量级：单层循环 $\rightarrow O(n)$ （穷举、求最大值、算校验和）；嵌套循环 $\rightarrow O(n^2)$ （冒泡、选择排序）；二分 $\rightarrow O(\lg n)$ （二分查找、遍历二叉树、找机器字里第一个置位）；分治 $\rightarrow O(n \lg n)$ （快排——注意它本名 $O(n^2)$ ，喂给它有序输入会退化，但平均是 $O(n \lg n)$ ）；组合 $\rightarrow$ 阶乘级（旅行商、装箱、集合划分——正是那些公认的难题）。

TIP 46

Test Your Estimates

After all this estimating, the only timing that counts is the speed of your code, running in the production environment, with real data.

TIP 46

检验你的估算

估算归估算，唯一算数的是你的代码在生产环境、用真实数据跑出来的速度。

解读 实践上：找到 $O(n^2)$ 的算法，试着改成分治降到 $O(n \lg n)$ ；不确定耗时/内存时，就实际跑，改变输入规

模、把结果画成曲线（三四个点就能看出形状）；别忘了实践因素——小规模输入下线性增长，喂进去几百万条时系统开始换页抖动，时间会突然恶化；用随机键测试排序很顺，第一次遇到有序输入可能就傻眼。作者还提醒「最快的不一定最好」：小数据集上简单插入排序跟快排一样快且更好写、好调；高启动成本的算法在小输入上可能得不偿失。警惕过早优化——先确认它真是瓶颈再花时间。

实践清单

- 看到循环就下意识估量级：外层嵌套想想 n 能长到多大。
- $O(n^2)$ 尽量找分治的 $O(n \lg n)$ 替代。
- 用真实数据、真实规模实测并画曲线，别只信理论。
- 别过早优化；也别忘了「最快的不一定最适合」。

33. Refactoring

重构 (TIP 47)

核心论点

程序演进过程中，重新审视早先的决定、改写部分代码是完全自然的——代码需要演化，它不是一个静态的东西。作者批评「把软件开发比作建筑施工」这个最常见的隐喻：那意味着「建筑师画图 → 承包商建造 → 住户入住并从此幸福生活」。软件不是这样。

Change and decay in all around I see... —H. F. Lyte, 环顾四周，我见到的尽是变化与衰败……—H. F. 莱特 “Abide With Me”

解读 作者主张：软件更像园艺而非建筑。你按计划和条件种下许多东西，有的茂盛、有的注定成堆肥；你会为了利用光影风雨而挪动植株，修剪疯长的，把撞色的移到更顺眼的地方，除草、施肥，持续观察花园的健康并随时调整。「建筑」的隐喻让管理者舒服（更科学、可重复、有清晰的汇报层级），但我们不是在建摩天大楼——我们不受物理和现实世界的边界那么强的约束。改写、返工、重构架构，统称为重构。

TIP 47

Refactor Early, Refactor Often

Keep track of the things that need to be refactored. If you can't refactor something immediately, make sure that it gets placed on the schedule.

TIP 47

尽早重构，经常重构

记录下需要重构的地方；若不能立刻重构，务必把它排进日程。

解读 何时重构？遇到以下情况就别犹豫：重复（违反 DRY）、非正交的设计、知识过时（需求漂移、理解加深）、性能（需要把功能挪个地方）。作者用医疗类比说服老板：「把需要重构的代码看作一个肿瘤。现在动刀，它还小；等它长大扩散再切，既更贵也更危险；再拖下去，可能连病人都保不住。」「没时间重构」这个借口站不住脚——现在不做，将来依赖更多时修复的代价更大。

解读 怎么重构？Martin Fowler 的三条建议：不要同时重构和加功能；动手前先有好的测试，并尽可能频繁地跑（这样出问题能马上知道）；每次迈小而审慎的一步（很多局部小改动累积成一次大改动，每步都测，就不会陷入漫长调试）。测试是「有底气重构」的关键。还有个小技巧：让破坏性的改动（改接口、改不兼容的功能）让构建失败，这样旧调用方编译不过，你能快速找出来一并更新。最后回到「软件熵」：别容忍破窗。

实践清单

- 发现重复/非正交/过时/性能问题，就重构（尽早、经常）。

- 重构和加功能不要同时做。
- 先有覆盖良好的测试，再动手；每小步跑一次测试。
- 破坏性改动让它「编译失败」，逼出所有旧调用方。

34. Code That's Easy to Test

易于测试的代码 (TIP 48、49)

核心论点

「软件 IC」是个常被提起的隐喻——软件组件应像集成电路芯片一样组合。但这只在组件 确实可靠时才成立。而芯片天生就是为测试而设计的：不只在出厂时、安装时，部署到现场后也要测（内建自测 BIST、测试访问机制 TAM）。软件也应如此：从最开始就把 可测试性设计进去，接起来之前先把每块测透。

TIP 48

Design to Test

When you design a module, or even a single routine, you should design both its contract and the code to test that contract.

TIP 48

为测试而设计

设计一个模块、甚至一个例程时，你既要设计它的契约，也要设计验证该契约的测试代码。

解读 单元测试相当于硬件里的芯片级测试。作者提出「针对契约测试」：把单元测试看作在验证该单元是否履行契约——这既告诉你代码是否满足契约，也告诉你契约本身是否如你所想。以 `sqrt` 为例，它的契约直接告诉我们该测什么：传负参应被拒绝、传 0 应被接受（边界值）、传 (0, 最大值] 区间的值应满足误差条件。测试顺序也讲究：若模块 A 用了 `LinkedList` 和 `Sort`，应先完整测 `LinkedList`、再完整测 `Sort`、最后测 A；这样一旦 A 失败而前两者通过，就能快速定位问题在 A（或其使用方式），省下大量调试时间。目的是避免造出「定时炸弹」——那些潜伏到项目后期才在尴尬时刻爆炸的东西。

TIP 49

Test Your Software, or Your Users Will

All software you write will be tested—if not by you and your team, then by the eventual users.

TIP 49

你不测试，用户就会替你测试

你写的所有软件终将被测试——若不是你和团队测，那就是最终用户来测。

解读 实践细节：测试不要塞在源码树角落——找不到就不会被用，小项目嵌入模块、大项目放子目录。测试本身还能充当用法示例和回归测试的基础。Java 里每个类都可以有自己的 `main` 跑单元测试（发布时被忽略，好处是出厂代码里仍带着测试）；C++ 里用 `#ifdef` 选择性编译测试。写多了就该有统一的 测试框架（harness），应具备：标准的 `setup/cleanup`、选择单个或全部测试、分析输出是否符合预期、标准化的失败报告，且测试要可组合（xUnit/JUnit 即为此而来）。调试时临时写的测试（ad hoc testing）别扔掉——代码坏过一次就可能再坏，要把它固化进单元测试。生产环境里的 bug 往往更刁钻，作者建议留一些「窥视内部状态」的手段：格式一致、可自动解析的日志；热键弹出诊断窗；甚至内嵌一个 Web 服务器用浏览器看内部状态。最后点出：测试更多是文化而非技术——「Perl 社区出人意料地重视测试」就是例子。

实践清单

- 写模块时连契约一起写测试；把边界条件列出来逐一测。
- 测试顺序：先测子组件，再测依赖它们的模块。
- 调试时临时写的测试要固化进单元测试，不要丢。

- 建统一的测试框架，测试可组合、可自动跑。
- 给生产环境留可解析的日志/诊断入口。

35. Evil Wizards

邪恶的向导 (TIP 50)

核心论点

应用越来越难写，界面越来越复杂。工具厂商于是抛出「银弹」——向导 (wizard)。需要带 OLE 容器支持的 MDI 应用？点一个按钮、答几个问题，向导自动生成骨架代码 (Visual C++ 一个场景就能生成 1200 多行)。向导确实好用。

TIP 50

Don't Use Wizard Code You Don't Understand

Unless Joe actually understands the code that has been produced on his behalf, he's fooling himself. He's programming by coincidence.

TIP 50

不要用你不理解的向导代码

除非 Joe 真正理解那些替他生成的代码，否则他是在自欺欺人——他正在靠巧合编程。

解读 作者的论点：用大师设计的向导，并不会让你自动变成大师。Joe 看着生成的一大堆代码和挺唬人的程序，觉得自己很行，只管往里加业务逻辑就准备发布——但只要他不理解这些代码，就失去了对应用的控制，将来改不动、也不好调。向导是单行道：它替你切好代码就走了，一旦生成的代码不对、或情况变了需要改，就只能靠你自己。作者强调他们并不反对向导（他们自己就写了一节讲「写你自己的代码生成器」），反对的是用你不理解的向导代码。有人反驳说「开发者本来就在依赖不懂的东西」（芯片的量子力学、CPU 中断结构、进程调度算法、库代码）——作者同意，但区别在于：那些是藏在整洁接口之后的库调用/系统服务，而向导生成的代码是一行行交织进 Joe 自己的应用里的，最终它不再是「向导的代码」，而成了「Joe 的代码」。谁都不该生产自己不完全理解的代码。

实践清单

- 用向导生成骨架后，逐行读懂它——读不懂就别依赖它。
- 宁可自己写（哪怕少一点），也不要接手看不懂的生成代码。
- 分清「藏在接口后的库」与「交织进你代码里的向导产物」。

第 6 章导读完。TIP 44–50 的完整标题对照见附录 A。

Before the Project

项目之前

No matter how well thought out it is, and regardless of which “best practices” it includes, no method can replace thinking.

无论考虑得多周全、包含了哪些「最佳实践」，没有哪种方法能取代思考。

动手之前先把基本规则立好，否则项目可能还没开始就已经注定失败。本章按时间顺序讲项目启动前的关键问题：怎么挖掘需求、怎么破解看似无解的难题、什么时候该听从「再等等」的直觉、为什么过度规约会变成陷阱，以及为什么不该被形式化方法和昂贵工具牵着走。

36. The Requirements Pit

需求陷阱 (TIP 51–54)

核心论点

很多教程把需求收集说成项目的早期阶段。「收集」(gathering) 一词暗示有一群快乐的分析师，在田园交响曲中弯腰捡起散落四周的智慧金块——好像需求本来就在那儿，你只需捡起来装进篮子。可事实并非如此：需求很少浮在表面，它们通常埋在层层假设、误解与政治之下。

Perfection is achieved, not when there is nothing left to add, but when there is nothing left to take away.
—Antoine de Saint-Exupéry, 1939

完美不在于无可增，而在于无可减。——圣埃克苏佩里，1939

TIP 51

Don’t Gather Requirements—Dig for Them

Requirements rarely lie on the surface. Normally, they’re buried deep beneath layers of assumptions, misconceptions, and politics.

TIP 51

不要收集需求——去挖掘需求

需求很少浮在表面，它们通常深埋在层层假设、误解与政治之下。

解读 什么才算真正的需求？简单说，它是「某件需要被完成之事的陈述」。但难点在于分清需求、策略与实现。用户可能说「只有员工的上级和人事部门才能查看该员工记录」——这其实是把业务策略写成了绝对陈述，而策略是会变的。作者的建议是：把这类策略单独记录并与需求超链接；需求写成通用陈述（「只有经授权的用户才能访问员工记录」），策略作为实现时需支持的例子。微妙之差后果却重大：前者逼开发者每次访问都硬编码一个判断，后者则引导你设计一套访问控制系统，策略变更时只需改那套系统的元数据。界面上的区别同样重要：「系统必须让你选择贷款期限」是需求；「需要一个列表框来选择」未必是——除非用户确实非列表框不可。

Requirements are not architecture. Requirements are not design, nor are they the user interface. Requirements are need.

需求不是架构，不是设计，也不是用户界面。需求就是需要。

TIP 52**Work with a User to Think Like a User**

Become a user...Spend a couple of days monitoring the phones with an experienced support person.

TIP 52**与用户共事，像用户一样思考**

去做一名用户……跟一位有经验的客服一起值几天电话班。

解读 一个常被忽视的技巧：成为用户。给客服写系统？去客服坐两天。在仓库待一周。除了让你看清系统真正会被怎么用，这句「我能跟着你上班一周吗？」还能建立信任、为沟通打下基础——注意别碍事就行。作者也强调要发现用户为什么这么做，而不只是他们目前怎么做；毕竟你的开发要解决的是他们的业务问题，而非仅仅满足其字面要求。文档方面，作者介绍 Ivar Jacobson 的用例（use case）概念和 Cockburn 的用例模板；并提醒别被 UML 的小人图（stick figure）误导——「不要做任何记号的奴隶，用最能向你的听众传达需求的方法」。

TIP 53**Abstractions Live Longer than Details**

It means generating statements such as: The system makes active use of an abstraction of DATEs.

TIP 53**抽象比细节活得更久**

它意味着写出这样的陈述：系统主动使用 DATE 的抽象。

解读 「向前看」不是要你预测未来，而是写出抽象。作者用千年虫（Y2K）为戒：那并不该怪程序员，真正的原因是没能超越当前业务实践加上违反 DRY。用两位年份是当时商业惯例，最早的自动化程序只是照搬了这个习惯；但即便输入/存储必须用两位数字，也本该有一个 DATE 抽象，知道「这两个数字是真实日期的缩写形式」。另外作者提醒警惕「需求蔓延」（requirements creep）：要跟踪需求变更（谁提的、谁批的、总共批了多少），把每个新功能对进度的影响当面指给项目发起人看——当项目延期一年、开始互相指责时，一份准确的「需求何时、如何膨胀」的记录会非常有用。

TIP 54**Use a Project Glossary**

...One place that defines all the specific terms and vocabulary used in a project.

TIP 54**使用项目术语表**

……一个集中的地方，定义项目中使用的一切特定术语与词汇。

解读 一开始讨论需求，用户和领域专家就会使用对他们有特定含义的词（比如区分「client」和「customer」）。要建立并维护项目术语表：它应广泛可访问（这正适合做成基于 Web 的文档）。作者说得很直白：如果用户和开发者用不同的词指同一件事，或者更糟——用同一个词指不同的事，这个项目就很难成功。把需求做成超文本文档还有个好处：不同读者各取所需（赞助人停留在高层抽象，程序员用超链接逐层下钻到细节）；也免得出现那种两英寸厚、没人看、一印出来就过时的《需求分析》活页夹——「放在网上，程序员或许才会读」。

实践清单

- 需求写成通用的语义陈述，把易变的策略单独记录并链接。
- 分清「需求 / 策略 / 实现 / 界面设计」，别把后者当前者。
- 亲自去做一段时间的「用户」，并追问「为什么这么做」。
- 建立并维护一词一义的项目术语表，放在人人可及处。
- 跟踪需求蔓延：记录每个新功能对进度的影响。

37. Solving Impossible Puzzles

破解无解的难题 (TIP 55)

核心论点

弗吕吉亚国王戈尔迪打了一个无人能解的结，神谕说解开者将统治亚洲。亚历山大来了，挥剑把结砍成碎片——不过是换了个解读需求的方式，而他最终确实统治了大部分亚洲。项目里也常冒出看似无解的难题：真的是那么难吗？

TIP 55

Don't Think Outside the Box—Find the Box

The key to solving puzzles is both to recognize the constraints placed on you and to recognize the degrees of freedom you do have.

TIP 55

别跳出框框想——先找到框框

解谜的关键，既在于认清加在你身上的约束，也在于认清你实际拥有的自由度。

解读 解「木制益智玩具」的方法：先别重复显而易见的解法，而是找出真正的约束。有些约束是绝对的（再讨厌、再蠢也必须遵守），有些只是先入之见。作者举了个老酒吧把戏：拿一瓶没开过的香槟打赌你能用它喝啤酒——窍门是把瓶子倒过来，往瓶底凹陷处倒一点啤酒。很多软件难题也这么刁钻。「跳出框框思考」这个流行说法并不准确：如果「框框」就是约束与条件的边界，那么真正的技巧是找到框框——它可能比你想象的大得多。

解读 作者的实操建议：面对貌似无解的问题，列出所有可能的路径——再荒谬、再蠢也别急着排除；然后逐条追问「这条路为什么走不通？你确定吗？能证明吗？」（特洛伊木马就是个「新颖解」：把军队弄进围墙之城又不想被发现，正门大概一开始就被当作自杀式方案排除了）。接着对约束分类并排序，像木工先切最长的料一样：先找出限制性最强的约束，再让其余约束落在它之内。最后是那串关键自问：有没有更简单的办法？你在解决对的问题吗，还是被某个枝节带偏了？这为什么是个问题？是什么让它这么难？非得这么做吗？非得做吗？很多时候，重新解读需求就能让一整批问题消失——就像戈尔迪之结。

实践清单

- 列出所有可能路径，包括看似荒谬的；逐条追问「为什么不行、能证明吗」。
- 区分「绝对约束」与「先入之见」，先满足限制最强的那条。
- 卡住时间：非得这么做吗？非得做吗？是不是在解错的问题？

38. Not Until You're Ready

没准备好就别开始 (TIP 56)

核心论点

顶尖表演者有个共同点：知道何时该开始、何时该等待。跳水者站在高台上等最佳起跳时刻，指挥家在乐团前举着双臂、直到感觉时机对了才落下第一拍。你也是顶尖表演者——也要听从那个低语「等等」的声音。坐下来准备敲键盘时若心里有隐隐的不安，就听它。

He who hesitates is sometimes saved.
Thurber, "The Glass in the Field"

—James

犹豫不决的人，有时反而得救。

—詹姆斯·瑟伯

TIP 56**Listen to Nagging Doubts—Start When You're Ready**

When you feel a nagging doubt...heed it. You may not be able to put your finger on exactly what's wrong, but give it time and your doubts will probably crystallize into something more solid.

TIP 56**倾听内心的疑虑——准备好了就开始**

当你感到隐隐不安时……听从它。你也许说不出到底哪里不对，但给它时间，疑虑多半会凝结成更具体、可着手处理的东西。

解读 「内在网球」式的训练：反复击球并对落点做语言化的反馈，让潜意识与反射在不知不觉中进步。你整个职业生涯都在做类似的事——积累经验与智慧。软件开发至今还不算一门科学，让你的直觉也参与进来。难点在于区分好判断与拖延：谁都怕白纸。作者的办法是开始做原型——挑一个你觉得困难的领域做概念验证。通常会有两种结果：其一，开始不久你就觉得浪费时间，这份无聊大概说明最初的迟疑只是想拖延——那就放弃原型、直接进入正式开发；其二，原型推进中你突然领悟某个基本前提错了，并且清楚看到了改正方法——那你的直觉是对的，果断丢掉原型、正式开工，还替团队省下了大量无用功。用原型来探索不安时，务必记住你为什么在做它（最糟的是几周后才想起「我本来是在做原型啊」）。作者还调侃：宣布「我去做个原型」在政治上往往也比「我不想开始」然后打开纸牌游戏体面得多。

实践清单

- 开工前有挥之不去的疑虑，先别硬上——它多半指向真问题。
- 用原型来区分「好判断」与「拖延」：很快觉得无聊就去干活，突然悟到前提错了就照直觉走。
- 用原型探路时，始终提醒自己这是原型，别变成正式开发。

39. The Specification Trap

规格说明的陷阱（TIP 57）

核心论点

程序规格说明，是把需求逐步细化到「程序员的手艺可以接手」的过程，本质上是一种沟通——解释并澄清，以消除主要歧义。它既是对后续维护者的记录，也是对用户的协定。「写规格说明是一项沉甸甸的责任。」

解读 问题在于很多设计者停不下来，觉得非要把每个细节都钉死才算对得起工资。作者指出几个原因这不对：第一，天真——以为规格能捕捉系统的每个细节与微妙之处；即便有形式化方法，仍需设计者向用户解释记号的含义，人的解读照样会出错；而且多数用户事先并不知道确切要什么，他们可能在你 200 页的文档上签了字，但一旦看到运行的系统，变更请求就会把你淹没。第二，自然语言本身的表达力有限——所有图表与形式化方法最终还得靠自然语言描述要执行的操作，而自然语言并不胜任（看看任何合同是如何被律师扭曲语言的）。作者出了个挑战：写一段话教别人系鞋带。你会发现「现在用拇指和食指捻一下，让绳头从左边那条鞋带下面穿过去……」附近就卡住了——这事难得出奇，可我们大多数人却能不假思索地系鞋带。

TIP 57**Some Things Are Better Done than Described**

A design that leaves the coder no room for interpretation robs the programming effort of any skill and art.

TIP 57**有些事做胜过说**

一个不给编码者留任何解释空间的设计，剥夺了编程工作中所有的技巧与艺术。

解读 第三是紧身衣效应。过度规定的设计会让程序员错过很多机会：编码时你常会发现「因为我是这样写的，

几乎不费劲就能加个额外功能」或「规格说要这样做，但换个做法结果几乎一样、还能省一半时间」。有这种过度规定，你连机会都看不到。作者主张把需求收集、设计、实现看作同一过程（交付高质量系统）的不同侧面，采取无缝的方式，让它们直接流入下一步、没有人为边界，并让实现与测试的反馈回流到规格过程。他并不反对写规格（有些场合——合同、特定环境、产品性质——确实需要极详细的规格），只是提醒：细节越多，回报递减甚至为负；也别在毫无实现或原型支撑的情况下层层堆叠规格——那样极易规定出根本造不出来的东西。最后警告：规格一旦变成保护开发者免受「写代码这个可怕世界」的安全毯，就越难走出来；团队若都裹在温暖舒服的规格里，就把他们拉出来——去看原型、或考虑曳光弹式开发。该开始写代码了！

实践清单

- 规格适可而止：达到清晰所需即可，别追求穷尽细节。
- 别在无实现/原型支撑时层层叠加规格——先做出来看。
- 让实现与测试的反馈回流到规格；警惕规格变成逃避编码的安全毯。

40. Circles and Arrows

圆圈与箭头（TIP 58、59）

核心论点

从结构化编程、主程序员团队、CASE 工具、瀑布模型、螺旋模型，到 Jackson、ER 图、Booch 云、OMT、UML……计算界从不缺「让编程更像工程」的方法。每种方法都招来信徒、红极一时，然后被下一种取代。作者形容一些开发者像海难幸存者抓住漂来的木头：每漂来一块新的就拼命游过去，指望它更好——可无论木头多好，人依旧在原地漫无目的地漂流。

[with] circles and arrows and a paragraph on the back of each one explaining what each one was, to be used as evidence against us... —Arlo Guthrie, “Alice’s Restaurant”

（照片上）画着圆圈和箭头，每张背面还写段话说明那是什么，好拿来当我们（受审）的证据…… —阿洛·格思里《爱丽丝的餐馆》

TIP 58

Don’t Be a Slave to Formal Methods

Never become a slave to a methodology: circles and arrows make poor masters.

TIP 58

不要做形式化方法的奴隶

永远别做方法论的奴隶——圆圈和箭头不是好主子。

解读 形式化方法的几处硬伤：多数用图表 + 少量文字捕捉需求，这些图表达的是设计者的理解，往往对最终用户毫无意义，于是设计者得去解释——用户并没有真正在「形式化地」校验需求，一切仍基于设计者的说明；作者更倾向于给用户看原型让他们玩。其次，形式化方法鼓励分工割裂——一组设计数据模型、一组看架构、一组收用例——他们见过这导致沟通不畅与浪费，还容易退回「设计者 vs 编码者」的对立心态；作者更愿意理解系统的整体。第三，作者喜欢可适配的动态系统（用元数据在运行时改变应用特性），而多数形式化方法把静态对象/数据模型与某种事件图结合，没看到一种能表达他们所需的动态性——反而会误导你在本该动态编织的对象之间建立静态关系。

解读 「方法真的划算吗？」Robert Glass 在 1999 年的一篇综述里回顾了七种技术（4GL、结构化技术、CASE、形式化方法、净室方法、过程模型、面向对象）的生产率与质量改善，结论是：最初的宣传都言过其实——即便有收益迹象，也往往要先经历显著的生产率与质量下降（引入与培训期）。别低估采用新工具/方法的成本，做好把头几个项目当作学习经历的准备。

TIP 59**Expensive Tools Do Not Produce Better Designs**

Try not to think about how much a tool cost when you look at its output.

TIP 59**昂贵的工具不产生更好的设计**

看一个工具的输出时，别去想它花了多少钱。

解读「我们该用形式化方法吗？」作者的答案出人意料地干脆：「当然该。但永远记住，形式化开发方法只是工具箱里的又一件工具。」若你仔细分析后觉得需要，就拥抱它——但要记住谁才是主人。务实程序员批判地看待方法论，从每种里汲取最好的部分，融合成一套每个月都在变好的工作实践；要不断打磨、改进流程，绝不要把一个方法论的僵硬边界当作你世界的边界，也别屈服于方法的虚假权威——别人可能带着一英亩的类图和 150 个用例来开会，但那堆纸仍只是他们对需求与设计可能出错的解读。最后作者点破那种「类图即应用、其余只是机械编码」的项目哲学：遇到它，你就该知道，你面对的是一支正在沉没的团队，和一段划不回家的漫漫水路。

实践清单

- 把形式化方法当工具箱里的一件工具，而非主人；批判性地取其精华。
- 别用一堆类图/用例来代替思考，也别被「方法的权威」吓住。
- 引入新方法前，预留学习期的生产率下降，并衡量成本。
- 看设计输出时，忘掉工具的价格。

第 7 章导读完。TIP 51–59 的完整标题对照见附录 A。

Pragmatic Projects

务实的项目

As your project gets under way, we need to move away from issues of individual philosophy and coding to talk about larger, project-sized issues.

当项目开始推进，我们就得从个人的哲学与编码问题，转向更宏观的、项目层面的问题。

本章不再讲个人技巧，而讲几个决定项目成败的关键领域：多人协作（务实团队）、让流程自动可靠（自动化）、项目级的测试哲学（无情测试）、文档（都是写作）、管理期望，以及最后一条——为自己的作品署名。

41. Pragmatic Teams

务实团队（TIP 60）

核心论点

前面讲的务实技巧都服务于个人。它们适用于团队吗？答案是响亮的「是」——务实个人的优势，在一个务实团队里会成倍放大。作者把前几节的理念逐条「重述」到团队层面。

At Group L, Stoffel oversees six first-rate programmers, a managerial challenge roughly comparable to herding cats. —The Washington Post Magazine, 1985

在 L 组，Stoffel 管着六位一流程序员——这 managerial 挑战，大致相当于「放猫」。

解读 作者把前面各节重述到团队：不容破窗（质量是团队的事，团队必须共同承担产品质量责任，支持理解「不留破窗」理念的成员；设「质量官」把责任外包是荒谬的——质量只能来自每个成员）；温水青蛙（团队比个人更容易被「煮熟」：人们以为「别人会处理」「组长应该批过了」；可以指定一个「试水温的人」持续检查范围膨胀、工期缩短、新功能、新环境等原协议之外的变化，不必一概拒绝，但必须知道它在发生）；沟通（团队作为整体要向外界清晰发声；好团队有鲜明的个性，文档干净一致，「用一个声音说话」——但对内要鼓励激烈辩论；有个小技巧是给项目起个品牌名和 Logo，给它一个可依附的身份）；**DRY**（设「项目图书管理员」或各功能的「对接人」，用 groupware / 内部新闻组沉淀问答，避免重复劳动）。

TIP 60

Organize Around Functionality, Not Job Functions

Split your people into small teams, each responsible for a particular functional aspect of the final system.

TIP 60

围绕功能组织，而非职务

把人拆成小团队，每个团队负责最终系统的一个特定功能方面。

解读 传统团队按瀑布模型的职务分工：业务分析师、架构师、设计师、程序员、测试员、文档员……还隐含有一个等级（离用户越近越资深），极端情况下甚至有「编码者不许跟测试员说话」这种荒谬规定。作者认为这是错的：分析、设计、编码、测试本就是同一问题的不同视图，人为割裂会带来一堆麻烦——离实际用户好几层

的程序员，不可能理解自己代码被使用的语境，也就无法做出明智决策。作者主张围绕功能组织（功能不限于终端用例，数据库访问层、帮助子系统都算），用与划分代码相同的标准（凝聚、自足）来划分团队，并运用契约、解耦、正交等技巧，把团队整体与「变化」的影响隔离开。注意「功能划分」需要 负责任的开发者和强有力的项目管理——项目至少要两个「头」：一个技术负责人（定风格、派任务、仲裁、盯全局），一个行政负责人（排资源、盯进度、定优先级、对外沟通）。

实践清单

- 按功能（而非职务）把团队拆成小、自足、可自主组织的单元。
- 设「试水温的人」盯住范围膨胀等变化；设图书管理员/对接人防重复。
- 对外用「一个声音」沟通，对内鼓励激烈辩论。
- 给项目起个名字和 Logo，建立团队身份。

42. Ubiquitous Automation

无处不在的自动化（TIP 61）

核心论点

文明因「把越来越多重要操作变成不用思考就能完成」而进步。汽车诞生之初，发动一辆 Model-T 福特要两页多的说明；现代汽车你只需拧钥匙——发动过程自动且万无一失。一个人照说明书操作可能把发动机淹了，自动启动器却不会。计算行业虽还处在「Model-T 阶段」，但我们也承受不起为常见操作反复走两页说明书。无论是构建发布、代码评审的文书，还是任何重复任务，都必须自动化。

Civilization advances by extending the number of important operations we can perform without thinking.
—Alfred North Whitehead

文明的进步，在于把越来越多重要操作变成无需思考即可完成。
——怀特海

TIP 61

Don't Use Manual Procedures

People just aren't as repeatable as computers are. Nor should we expect them to be.

TIP 61

不要用手工流程

人就是不像计算机那样可重复，我们也不该指望人做到。

解读 作者讲了个真实案例：客户现场所有开发者用同一 IDE，系统管理员发了一份好几页的「安装附加包」说明（点这儿、滚那儿、拖一下、双击那个……）。结果每台机器略有差异，同一份代码在不同机器上表现出细微不同：bug 在一台机器出现、其他机器没有。手工流程把一致性交给了运气。而脚本会不厌其烦地按同样顺序执行同样的指令，还能纳入版本控制（这样「它以前是好的……」也能查）。自动化的好帮手：cron（定时无人值守地跑备份、夜间构建、站点维护）；用 makefile 编译（可插入代码生成钩子、自动跑回归测试，目标是「一条命令完成检出、构建、测试、发布」）；构建自动化（从空目录从零构建，标版本号、生成可分发的镜像、跑测试，通常每晚自动跑一遍完整测试——这样一旦回归测试失败，你能就近发现并修复，而不是三个月后才发现问题坏了）；最终构建（make final）。还有「自动化的日常文书」：夜间构建自动发布文档到内部网站（这是 DRY 的又一次应用——Web 上的呈现只是一个视图）；用源码里的状态标记（如 Status: needs_review）自动驱动代码评审的排期与审批。最后是那句「鞋匠的孩子没鞋穿」——写软件的人常用最差的工具干活；而我们其实有 cron、make、脚本语言这些现成材料。让计算机去做重复、枯燥的事吧，它比我们做得更好。

实践清单

- 任何重复性操作（构建、发布、评审文书）都写成脚本，纳入版本控制。
- 用 cron 定时跑备份、夜间构建、站点维护。
- 一条命令完成「检出 → 构建 → 测试 → 发布」。
- 每晚跑完整测试；文档/网站从仓库自动生成，不手工维护。
- 别让「鞋匠的孩子没鞋穿」——给自己做工具。

43. Ruthless Testing

无情的测试（TIP 62–66）

核心论点

多数开发者讨厌测试，于是温柔地测——潜意识里知道哪里会坏、就避开那片区域。务实程序员不一样：我们主动现在就把 bug 找出来，免得日后被别人找出来时丢脸。找 bug 像用网捕鱼：细密的网（单元测试）捞小鱼，粗大的网（集成测试）捕食人鲨。

TIP 62

Test Early. Test Often. Test Automatically.

The earlier a bug is found, the cheaper it is to remedy.

TIP 62

早测试，常测试，自动化测试

bug 发现得越早，修复成本越低。

TIP 63

Coding Ain't Done 'Til All the Tests Run

Just because you have finished hacking out a piece of code doesn't mean you can go tell your boss or your client that it's done. It's not.

TIP 63

所有测试都跑通，编码才算完

敲完一段代码，不等于你就可以去告诉老板或客户「做完了」。还没完。

解读 测什么：单元测试（其余一切测试的基础）、集成测试（子系统之间是否协同——它往往是系统 bug 最大的来源）、验证与确认（「用户说要这个，但这真是他们需要的吗？」——没有 bug 却答错问题的系统毫无用处）、资源耗尽/错误/恢复（真实世界里程序没有无限资源：内存、磁盘、CPU、带宽、色彩、分辨率……它会「优雅地失败」吗，还是当着用户的面 core dump?）、性能测试（预期并发/负载下是否达标、是否可扩展）、可用性测试（真实用户、真实环境；不满足可用性标准和除以零一样是 bug）。

解读 怎么测：回归测试（对比当前输出与已知值，确保今天修的 bug 没弄坏昨天的功能——重要的安全网）；测试数据（真实数据与合成数据都要，各有各能暴露的问题——合成数据可用于压边界、造统计特性）；GUI 测试（工具化，但并非一切都能自动化，比如刻意非确定性的视觉效果只能人工判读；解耦设计让你能在无 GUI 情况下测业务逻辑）；测试你的测试；设计/方法学度量（KK 圈复杂度、继承扇入扇出、响应集、类耦合比——用统计方法找「显著偏离」的模块）。

TIP 64

Use Saboteurs to Test Your Testing

After you have written a test to detect a particular bug, cause the bug deliberately and make sure the test complains.

TIP 64

用「破坏者」来测试你的测试

写好了检测某个 bug 的测试后，故意制造那个 bug，确认测试真的会报警。

TIP 65**Test State Coverage, Not Code Coverage**

Even if you do happen to hit every line of code, that's not the whole picture. What is important is the number of states your program may have.

TIP 65**测试状态覆盖，而非代码覆盖**

即便你命中了每一行代码，那也不是全貌。重要的是程序可能处于的状态数量。

解读 覆盖率工具只能给你大致感觉，别指望 100%。作者的例子极有说服力：`int test(int a,int b){ return a/(a+b); }` 只有三行，理论上却有 100 万个逻辑状态，其中 999999 个正确、1 个会崩 (`a+b==0`)。「执行过这行代码」并不能告诉你这一点。而完整枚举状态一般是不可能的难题——难到「太阳都变成冷硬的一团了你还解出来」。更关键的是：遍历代码的顺序，影响往往比覆盖率更大。

TIP 66**Find Bugs Once**

Once a human tester finds a bug, it should be the last time a human tester finds that bug.

TIP 66**同一个 bug 只找一次**

一旦人发现了一个 bug，这个 bug 就应该是最后一次由人来发现。

解读 这是作者认为测试中最重要的概念——显而易见、几乎每本教材都这么说，可多数项目仍然做不到：如果一个 bug 从现有测试网里漏过去了，就必须补一个新测试把它兜住。此后每次都要自动检查这个 bug，「无例外、不论多琐碎、不论开发者怎么抱怨『这不会再发生了』」——因为它就是会再发生。而我们没有时间去追那些自动测试本可以替我们找到的 bug；我们的时间要用来写新代码——和新 bug。

实践清单

- 有代码就开测，测试代码常比生产代码还多；测试结果也要自动判读。
- 修好 bug 的同时补上能捕获它的回归测试（Find Bugs Once）。
- 写完测试后故意制造该 bug，验证测试会报警（破坏者）。
- 关注状态覆盖与遍历顺序，别迷信代码覆盖率。
- 压力/性能等重测试可定期跑（周/月），但必须排进日程。

44. It's All Writing

一切都是写作（TIP 67、68）

核心论点

开发者通常不重视文档：好一点把它当作「不幸的必要」，差一点当成低优先级任务，指望管理层在项目末期忘了它。务实程序员把文档当作开发过程不可分割的一部分。作者要淡化「代码 vs 文档」的二分，把它们看作同一模型的两个视图，并把务实的原则同样应用到文档上。

The palest ink is better than the best memory.
—Chinese Proverb

好记性不如烂笔头。

—中国谚语

TIP 67**把英语当成另一种编程语言**

所有文档都是代码的一面镜子。若两者不一致，代码才算数。

TIP 67**Treat English as Just Another Programming Language**

All documentation is a mirror of the code. If there's a discrepancy, the code is what matters.

解读 文档分内部（源码注释、设计/测试文档）与外部（用户手册等），但无论读者是谁、作者是开发者还是技术写手，文档都是代码的镜像。关于代码注释：注释应说明为什么做（目的、目标），而不是如何做——「如何」代码已经写明，重复它是违反 DRY 的。注释恰好适合记录那些别处无处安放的东西：工程权衡、决策原因、被放弃的备选方案。命名要达意（`foo`、`stuff`、`manager` 无意义；OO 里用匈牙利命名很糟），有误导性的名字比无意义的名字更糟（有个叫 `getData` 的例程其实在写数据——人的大脑会反复被这种不一致绊倒，这叫 Stroop 效应）。源码注释里不该出现：导出函数清单、修订历史、依赖文件清单、文件名——这些都能由工具自动维护；但作者/责任人的名字应当出现，把责任与问责挂到代码上大有好处。

TIP 68**Build Documentation In, Don't Bolt It On**

Your document is now an integral part of the project development... You are guaranteeing that the specification, schema, and code all agree.

TIP 68**把文档做进去，而不是补上去**

你的文档现在成了项目开发不可分割的一部分
……你保证了规格、*schema* 与代码三者始终一致。

解读 作者提出可执行文档：以前同一份信息（表结构）会重复三处——规格说明、建表的 SQL、保存一行数据的代码记录结构，改一处另两处就过时，明显违反 DRY。解法是选定权威来源（比如规格文档）作为模型，再自动导出成不同视图（数据库 *schema*、语言记录结构）。若文档是带标记的纯文本，用 Perl 之类抽取重排即可；若是私有格式，可以写宏、或让文档从属于另一个权威源。目标是永远只修改模型，所有视图自动更新。作者也提醒：别把资料「扔过墙」给技术写手——写手也应遵循 DRY、正交、模型-视图、自动化这些原则。关于印刷 vs Web：纸张文档一印出来就可能过时，应尽量做成可在线的、带超链接的形态并加时间戳/版本号；同一份内容要输出多种格式时，内容的呈现应与内容本身独立（用标记语言 + XSL/CSS，而非复制粘贴）。最后点题：文档与代码是同一底层模型的不同视图；「别让文档沦为二等公民」——用对待代码的认真劲对待文档，用户和后续维护者都会为你歌唱。

实践清单

- 注释写「为什么」，而非「如何」；用有意义的变量名，杜绝误导命名。
- 让工具自动维护导出函数清单、修订历史、文件名、版权声明。
- 选定唯一权威来源，其余形态（*schema*、代码、手册）自动生成。
- 文档与代码一并纳入版本控制、一并参与夜间构建。
- 内容与呈现分离：一份源标记输出印刷/Web/在线帮助/幻灯片。

45. Great Expectations

远大的期望 (TIP 69)

核心论点

一家公司宣布创纪录的利润，股价却跌了 20%——因为没达到分析师的预期。孩子打开昂贵的圣诞礼物却哭了——因为他想要的是那个便宜的娃娃。一个团队使出浑身解数实现了极其复杂的应用，却因缺少帮助系统而被用户冷落。抽象地讲，一个应用只要正确实现了规格就算成功；可惜这只够付抽象的账单。现实中，项目的成败取决于它多大程度上满足了用户的期望。

Be astonished, O ye heavens, at this, and be horribly afraid...
—Jeremiah 2:12

诸天哪，要因此惊奇，极其恐慌…… —《耶利米书》2:12

TIP 69

Gently Exceed Your Users' Expectations

If you work closely with your users...there will be few surprises when the project gets delivered. This is a BAD THING. Try to surprise your users. Not scare them, but delight them.

TIP 69

温和地超越用户的期望

若你与用户紧密合作……交付时就不会有什么意外。这其实是件坏事。试着给用户惊喜——不是惊吓，而是愉悦。

解读 用户带着某种愿景来找你——可能不完整、不一致、技术上不可能，但那是他的愿景，投入了情感（像圣诞节的孩子的期待），你不能置之不理。有些顾问把这叫「管理期望」（主动控制用户能指望得到什么），作者认为那有点精英主义：我们的职责不是控制用户的希望，而是与他们一起，对开发过程和最终交付物达成共识，包括那些他们还没说出口的期望。最有助于此的两项技术是曳光弹和原型——都让团队做出用户能看见的东西，是沟通理解、练习彼此交流的理想方式。

实践清单

- 全程与用户一起校准期望，包括他们还没说出口的。
- 用曳光弹/原型让用户尽早看到实物。
- 在期望之上温和地多做一点：ToolTip、快捷键、速查卡、彩色化、安装自动化、日志分析器、系统自检工具、定制启动画面……小小善意，回报丰厚。
- 加这些「惊喜」时，别把系统搞坏。

46. Pride and Prejudice

骄傲与偏见 (TIP 70)

核心论点

务实程序员不逃避责任，反而乐于接受挑战、乐于让自己的专业能力为人所知。若我们为某个设计或某段代码负责，就要做出一份自己引以为傲的工作。早年的工匠以在作品上署名为荣；你也应当如此。

You have delighted us long enough. —Jane Austen, *Pride and Prejudice* 你让我们高兴得够久了。——简·奥斯汀《傲慢与偏见》

TIP 70**Sign Your Work**

Your signature should come to be recognized as an indicator of quality.

TIP 70**为你的作品署名**

你的署名，应当逐渐成为品质的标志。

解读 署名也可能带来麻烦：代码所有权若处理不好，会让人变得有领地意识、不愿碰公共基础设施，项目碎成一堆小封地，你开始偏袒自己的代码、敌视同事的代码——那正是书名里的「偏见」。作者的意思不是这个：不要嫉妒地守护你的代码不让别人碰，同样，也要尊重别人的代码；「黄金法则」与开发者之间相互尊重，是这一条能奏效的前提。另一方面，匿名（尤其在大项目里）是马虎、错误、懒惰和烂代码的温床——太容易把自己当成一颗螺丝钉，在没完没了的状态报告里找蹩脚的借口，而不是写好代码。代码当然要被「拥有」，但不必由个人拥有——Kent Beck 的极限编程就主张共同拥有代码（不过需要结对编程等额外实践来对抗匿名带来的危险）。作者想要的是所有者的骄傲：「这是我写的，我为我的作品负责。」当人们看到你的名字，会期待那是一段扎实、优雅、经过测试、有文档的代码——一份真正专业的作品，出自一位真正的专业人士，一位 Pragmatic Programmer。

实践清单

- 为你负责的代码署名，并为它负责到底。
- 不要嫉妒地守护自己的代码，也别怠慢别人的代码（黄金法则）。
- 以「别人看到你的名字就信任这段代码」为目标。
- 若采用集体代码所有权，配套结对等实践，防止匿名带来的松懈。

第 8 章导读完。至此全书八章导读收官，TIP 1–70 的完整标题对照见附录 A。

TIP Cheatsheet TIP 速查卡

全书 70 条 tip 的一句话速记，按章分组，供快速回顾。完整标题对照见下一节。

第一章 务实的哲学 (TIP 1–10)

TIP 1 Care About Your Craft 关心你的手艺

不在意把事做好，就别做软件。

TIP 2 Think! About Your Work 思考！关于你的工作

做事时实时审视，别开「自动驾驶」。

TIP 3 Provide Options, Don't Make Lame Excuses 提供选项，别找蹩脚借口

出错时给方案，不给借口。

TIP 4 Don't Live with Broken Windows 不要容忍破窗

发现即修；一时修不好，先「钉木板」。

TIP 5 Be a Catalyst for Change 做变革的催化剂

先做小而可用的成品，再顺水推舟扩展。

TIP 6 Remember the Big Picture 记住大局

别被温水煮了——定期跳出来看全局。

TIP 7 Make Quality a Requirements Issue 把质量当作需求问题

「够好」由用户参与界定。

TIP 8 Invest Regularly in Your Knowledge Portfolio 定期投资知识资产

习惯比投入量更重要。

TIP 9 Critically Analyze What You Read and Hear 批判地分析所读所闻

多问一句：谁在从中获利？

TIP 10 It's Both What You Say and the Way You Say It 说什么与怎么说同样重要

对方没听进去 = 没在沟通。

第二章 务实的方法 (TIP 11–19)

TIP 11 DRY 不要重复自己

每份知识只留一份权威表示。

TIP 12 Make It Easy to Reuse 让复用变得容易

复用比自写更省事，人才会复用。

TIP 13 Eliminate Effects Between Unrelated Things 消除不相关事物的影响

追求正交：改动不蔓延。

TIP 14 There Are No Final Decisions 没有最终决定

决策写在沙滩上，别刻在石头上。

TIP 15 Use Tracer Bullets to Find the Target 用曳光弹寻找目标

端到端打通，换取即时反馈。

TIP 16 Prototype to Learn 用原型学习

价值在学到的东西，不在产出的代码。

TIP 17 Program Close to the Problem Domain 贴近问题领域编程

用领域的语言说话、报错。

TIP 18 Estimate to Avoid Surprises 估算以避免意外

单位暗示精度；先问用途与所需准确度。

TIP 19 Iterate the Schedule with the Code 让进度表随代码迭代

时间表靠本项目的经验定。

第三章 基础工具 (TIP 20–29)

TIP 20 Keep Knowledge in Plain Text 知识存纯文本

自描述，且任何工具都能处理。

TIP 21 Use the Power of Command Shells 善用命令行

一行管道常胜过十次点击。

TIP 22 Use a Single Editor Well 熟练一个编辑器

统一到肌肉记忆：手不离键盘。

TIP 23 Always Use Source Code Control 始终用版本控制

哪怕单人一周、哪怕不是源码。

TIP 24 Fix the Problem, Not the Blame 解决问题，不归咎于人

bug 是谁的错都不重要，它都是你的问题。

TIP 25 Don't Panic 不要惊慌

说「不可能」时，是你的某个假设错了。

TIP 26 “select” Isn't Broken select 没有坏

见蹄印先想马、别想斑马。

TIP 27 Don't Assume It—Prove It 别假设——去证明

惊讶程度正比于你对那段代码的信任。

TIP 28 Learn a Text Manipulation Language 学一门文本处理语言

用它试想法只要半小时，而非五小时。

TIP 29 Write Code That Writes Code 写能生成代码的代码

一份知识自动派生所有形态。

第四章 务实的偏执 (TIP 30–35)

TIP 30 You Can't Write Perfect Software 你写不出完美软件

接受它，然后转向防御。

TIP 31 Design with Contracts 用契约设计

前置/后置/不变式：接受严格，承诺极少。

TIP 32 Crash Early 尽早崩溃

死程序比癩程序危害小。

TIP 33 If It Can't Happen, Use Assertions 用断言守「不可能」

别在上线时抽掉安全网。

TIP 34 Use Exceptions for Exceptional Problems 异常只用于异常问题

别把它当常规控制流。

TIP 35 Finish What You Start 有始有终

谁分配谁释放；逆序释放，统一顺序。

第五章 弯曲，或折断 (TIP 36–43)

TIP 36 Minimize Coupling Between Modules 最小化模块间耦合

迪米特法则：别链式「伸手」。

TIP 37 Configure, Don't Integrate 配置，而非集成

会变的东西做成配置项。

TIP 38 Put Abstractions in Code, Details in Metadata 抽象进代码，细节进元数据

一般情形写引擎，具体情形放外面。

TIP 39 Analyze Workflow to Improve Concurrency 分析工作流以改善并发

找出本可并行却被串行的步骤。

TIP 40 Design Using Services 用服务来设计

独立并发的对象，藏在一致接口之后。

TIP 41 Always Design for Concurrency 始终为并发而设计

对象在任何被调用的时刻都得合法。

TIP 42 Separate Views from Models 视图与模型分离

同模型多视图，灵活性很便宜。

TIP 43 Use Blackboards to Coordinate Workflow 用黑板协调工作流

让参与者匿名、异步地交换数据。

第六章 编码之时 (TIP 44–50)

TIP 44 Don't Program by Coincidence 不要靠巧合编程

弄清楚它为什么能工作。

TIP 45 Estimate the Order of Your Algorithms 估算算法的量级

看到循环就估 O 。

TIP 46 Test Your Estimates 检验你的估算

唯一算数的是真实数据下的实测。

TIP 47 Refactor Early, Refactor Often 尽早重构，经常重构

像切肿瘤：早切便宜；动手前先有测试。

TIP 48 Design to Test 为测试而设计

契约与测试一起设计；先测子组件。

TIP 49 Test Your Software, or Your Users Will 你不测试，用户就会替你测试

测试是文化，不是技术。

TIP 50 Don't Use Wizard Code You Don't Understand 不要用你不理解的向导代码
交织进你代码的，就得你来负责。

第七章 项目之前 (TIP 51–59)

TIP 51 Don't Gather Requirements—Dig for Them 不要收集需求——去挖掘需求
分清需求、策略与实现。

TIP 52 Work with a User to Think Like a User 与用户共事，像用户一样思考
亲自去当一段时间的用户。

TIP 53 Abstractions Live Longer than Details 抽象比细节活得更久
千年虫的教训：抽象出 DATE。

TIP 54 Use a Project Glossary 使用项目术语表
一词一义，人人可及。

TIP 55 Don't Think Outside the Box—Find the Box 别跳出框框——先找到框框
区分绝对约束与先入之见。

TIP 56 Listen to Nagging Doubts 倾听内心的疑虑
用原型区分「好判断」与「拖延」。

TIP 57 Some Things Are Better Done than Described 有些事做胜过说
过度规定会扼杀技巧与机会。

TIP 58 Don't Be a Slave to Formal Methods 不要做形式化方法的奴隶
圆圈和箭头不是好主子。

TIP 59 Expensive Tools Do Not Produce Better Designs 昂贵的工具不产生更好的设计
看输出时，忘掉工具的价格。

第八章 务实的项目 (TIP 60–70)

TIP 60 Organize Around Functionality, Not Job Functions 围绕功能组织，而非职务
按功能拆成自足的小团队。

TIP 61 Don't Use Manual Procedures 不要用手工流程
脚本比人更可重复。

TIP 62 Test Early. Test Often. Test Automatically. 早测试，常测试，自动化测试

bug 发现越早，修复越便宜。

TIP 63 Coding Ain't Done 'Til All the Tests Run 测试全跑通，编码才算完

通过全部测试才叫「可用」。

TIP 64 Use Saboteurs to Test Your Testing 用「破坏者」测试你的测试

故意造 bug，确认测试会报警。

TIP 65 Test State Coverage, Not Code Coverage 测试状态覆盖，而非代码覆盖

行覆盖不等于状态覆盖。

TIP 66 Find Bugs Once 同一个 bug 只找一次

漏网的 bug，必须补测试兜住。

TIP 67 Treat English as Just Another Programming Language 把英语当成另一种编程语言

注释写「为什么」，不写「如何」。

TIP 68 Build Documentation In, Don't Bolt It On 把文档做进去，而不是补上去

唯一权威源，其余视图自动派生。

TIP 69 Gently Exceed Your Users' Expectations 温和地超越用户的期望

给惊喜，不给惊吓。

TIP 70 Sign Your Work 为你的作品署名

让名字成为品质的标志。

Tips Index TIP 总索引

#	English	中文
1	Care About Your Craft	关心你的手艺
2	Think! About Your Work	思考！关于你的工作
3	Provide Options, Don't Make Lame Excuses	提供选项，别找蹩脚的借口
4	Don't Live with Broken Windows	不要容忍破窗
5	Be a Catalyst for Change	做变革的催化剂
6	Remember the Big Picture	记住大局
7	Make Quality a Requirements Issue	把质量当作需求问题
8	Invest Regularly in Your Knowledge Portfolio	定期投资你的知识资产
9	Critically Analyze What You Read and Hear	批判性地分析你所读所闻
10	It's Both What You Say and the Way You Say It	说什么与怎么说同样重要
11	DRY - Don't Repeat Yourself	DRY——不要重复自己
12	Make It Easy to Reuse	让复用变得容易
13	Eliminate Effects Between Unrelated Things	消除不相关事物之间的影响
14	There Are No Final Decisions	没有最终决定
15	Use Tracer Bullets to Find the Target	用曳光弹寻找目标
16	Prototype to Learn	用原型学习
17	Program Close to the Problem Domain	贴近问题领域编程
18	Estimate to Avoid Surprises	估算以避免意外
19	Iterate the Schedule with the Code	让进度表随代码迭代
20	Keep Knowledge in Plain Text	让知识保存在纯文本中
21	Use the Power of Command Shells	善用命令行的力量
22	Use a Single Editor Well	熟练使用一个编辑器
23	Always Use Source Code Control	始终使用源码版本控制
24	Fix the Problem, Not the Blame	解决问题，而不是归咎于人
25	Don't Panic	不要惊慌
26	“select” Isn't Broken	“select” 没有坏
27	Don't Assume It - Prove It	别假设——去证明
28	Learn a Text Manipulation Language	学一门文本处理语言
29	Write Code That Writes Code	写能生成代码的代码
30	You Can't Write Perfect Software	你写不出完美的软件
31	Design with Contracts	用契约来设计
32	Crash Early	尽早崩溃
33	If It Can't Happen, Use Assertions to Ensure That It Won't	若不可能发生，就用断言确保它不会
34	Use Exceptions for Exceptional Problems	用异常处理异常问题
35	Finish What You Start	有始有终
36	Minimize Coupling Between Modules	最小化模块间的耦合

#	English	中文
37	Configure, Don't Integrate	配置，而非集成
38	Put Abstractions in Code, Details in Metadata	抽象进代码，细节进元数据
39	Analyze Workflow to Improve Concurrency	分析工作流以改善并发
40	Design Using Services	用服务来设计
41	Always Design for Concurrency	始终为并发而设计
42	Separate Views from Models	让视图与模型分离
43	Use Blackboards to Coordinate Workflow	用黑板协调工作流
44	Don't Program by Coincidence	不要靠巧合编程
45	Estimate the Order of Your Algorithms	估算算法的量级
46	Test Your Estimates	检验你的估算
47	Refactor Early, Refactor Often	尽早重构，经常重构
48	Design to Test	为测试而设计
49	Test Your Software, or Your Users Will	你不测试，用户就会替你测试
50	Don't Use Wizard Code You Don't Understand	不要用你不理解的向导代码
51	Don't Gather Requirements - Dig for Them	不要收集需求——去挖掘需求
52	Work with a User to Think Like a User	与用户共事，像用户一样思考
53	Abstractions Live Longer than Details	抽象比细节活得更久
54	Use a Project Glossary	使用项目术语表
55	Don't Think Outside the Box - Find the Box	别跳出框框想——先找到框框
56	Listen to Nagging Doubts - Start When You're Ready	倾听内心的疑虑——准备好了就开始
57	Some Things Are Better Done than Described	有些事做胜过说
58	Don't Be a Slave to Formal Methods	不要做形式化方法的奴隶
59	Expensive Tools Do Not Produce Better Designs	昂贵的工具不产生更好的设计
60	Organize Around Functionality, Not Job Functions	围绕功能组织，而非职务
61	Don't Use Manual Procedures	不要用手工艺流程
62	Test Early. Test Often. Test Automatically.	早测试，常测试，自动化测试
63	Coding Ain't Done 'Til All the Tests Run	所有测试都跑通，编码才算完
64	Use Saboteurs to Test Your Testing	用“破坏者”来测试你的测试
65	Test State Coverage, Not Code Coverage	测试状态覆盖，而非代码覆盖
66	Find Bugs Once	同一个 bug 只找一次
67	Treat English as Just Another Programming Language	把英语当成另一种编程语言
68	Build Documentation In, Don't Bolt It On	把文档做进去，而不是补上去
69	Gently Exceed Your Users' Expectations	温和地超越用户的期望
70	Sign Your Work	为你的作品署名